PARKING: SMART REAL-TIME PARKING MANAGEMENT AND RESERVATION SYSTEM

A PROJECT REPORT

Submitted by

NITISH R (Register No: 310624104000)

in partial fulfillment for the award of the Degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION)
AFFILIATED TO ANNA UNIVERSITY, CHENNAI
RAMAPURAM, CHENNAI – 600089

OCTOBER 2026

EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION)
RAMAPURAM, CHENNAI – 600089

BONAFIDE CERTIFICATE

Certified that this project report titled "PARKING: SMART REAL-TIME PARKING MANAGEMENT AND RESERVATION SYSTEM" is the bonafide work of NITISH R (Register No: 310624104000), who carried out the engineering project work under my supervision in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering during the academic year 2026–2027.

The project demonstrates an end-to-end cloud-native intelligent parking reservation infrastructure, incorporating real-time geospatial discovery, atomic concurrency controls, cryptographically verified QR ticket gating, and role-based operational dashboards for drivers, security guards, parking lot owners, and platform administrators.

HEAD OF THE DEPARTMENT

Dr. S. Cloudin, M.E., Ph.D.
Professor and Head
Department of Computer Science and Engg.
Easwari Engineering College
Ramapuram, Chennai – 600089

PROJECT SUPERVISOR

Ms. Swapna Duggi, M.Tech.
Assistant Professor
Department of Computer Science and Engg.
Easwari Engineering College
Ramapuram, Chennai – 600089

EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION)
RAMAPURAM, CHENNAI – 600089

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE OF VIVA-VOCE EXAMINATION

This is to certify that the Viva-Voce Examination of the Project Work entitled "PARKING: SMART REAL-TIME PARKING MANAGEMENT AND RESERVATION SYSTEM" submitted by Mr. NITISH R (Register No: 310624104000) in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering was conducted on 19.10.2026 in the Department of Computer Science and Engineering, Easwari Engineering College, Ramapuram, Chennai.

INTERNAL EXAMINER	EXTERNAL EXAMINER
_____	_____
Date:	Date:
Department of CSE	Department of CSE
Easwari Engineering College	Anna University / Nominated Institution

DECLARATION

I, NITISH R (Register No: 310624104000), hereby declare that the project work entitled "PARKING: SMART REAL-TIME PARKING MANAGEMENT AND RESERVATION SYSTEM" is an authentic record of independent work carried out by me under the guidance of Ms. Swapna Duggi, Assistant Professor, Department of Computer Science and Engineering, Easwari Engineering College, Chennai. This project report has not previously formed the basis for the award of any degree, diploma, associate-ship, or other similar title to any candidate.

NITISH R
(310624104000)
Place: Chennai
Date: 19.10.2026

ACKNOWLEDGEMENT

I express my profound gratitude to the Principal, Dr. P. Deiva Sundari, and the Management of Easwari Engineering College for their unwavering encouragement and for providing the modern computing and network infrastructure essential for carrying out this research and development work.

I express my heartfelt thanks to the Head of the Department, Dr. S. Cloudin, Department of Computer Science and Engineering, for his visionary support, insightful critique, and sustained motivation throughout the project formulation.

I am deeply indebted to my Project Supervisor, Ms. Swapna Duggi, Assistant Professor, Department of Computer Science and Engineering, whose rigorous technical guidance, meticulous feedback, and domain expertise were instrumental in navigating architectural, concurrency, and security challenges encountered throughout the project lifecycle.

I extend my sincere thanks to all faculty members, technical laboratory assistants, and staff of the Department of Computer Science and Engineering for their continuous co-operation. Finally, I express my warmest thanks to my family and friends for their constant patience, understanding, and steadfast moral support during the completion of this engineering report.

ABSTRACT

Urban vehicular congestion represents a critical socioeconomic challenge in modern metropolitan environments, with empirical studies indicating that searching for vacant parking spaces accounts for over 30% of downtown traffic congestion, massive fuel wastage, and elevated carbon emissions. Conventional parking systems remain heavily reliant on static on-site signage, manual attendant ticketing, uncoordinated drive-ins, and fragmented proprietary hardware. These traditional approaches fail to provide advance booking guarantees, lack real-time synchronization, and suffer from high rates of double-booking conflicts, revenue leakage, and inefficient lot utilization.

To mitigate these inefficiencies, this project presents ParKING, an end-to-end, reactive smart parking management and reservation platform. ParKING bridges the gap between drivers seeking assured parking spots and facility managers governing parking assets. The system is engineered as a modern full-stack web application leveraging React 19, TypeScript, Vite, TanStack Router, Tailwind CSS 4, and Lucide Icons on the client tier, coupled with a modular Node.js Express 5 REST API gateway and a real-time WebSocket messaging layer orchestrated via Socket.IO. Data persistence is managed through MongoDB with Mongoose 9.1 ODM, employing a high-performance 2dsphere geospatial index to execute sub-second spherical queries ($near) that discover vacant parking lots within a 5-kilometer commuter radius.

A primary technical innovation of ParKING is its atomic concurrency-safe slot allocation engine. By utilizing atomic findOneAndUpdate operations with precondition checks, the platform guarantees zero race conditions or double-booking occurrences even under intense concurrent driver demand. Furthermore, the system accommodates both replica-set clusters and standalone local MongoDB topologies through an intelligent transaction-detection and compensation-rollback mechanism (withTransaction). Digital access is secured through cryptographically generated opaque 32-byte hexadecimal tokens, rendered as QR codes and companion human-readable identifiers (PK-XXXXXXXX), eliminating the exposure of sensitive driver identity data.

The operational lifecycle encompasses dedicated role-based portals for Drivers, Security Guards, Parking Facility Owners, and Platform Administrators. Guards utilize a mobile browser camera scanner powered by html5-qrcode to validate entrance and departure in real time, triggering automatic duration-based fee calculations and broadcasting live occupancy snapshots across WebSocket rooms. A dedicated IoT hardware simulator endpoint enables hardware sensors to push ultrasonic space telemetry while preserving reservation integrity. Comprehensive automated verification comprising 12 backend unit tests and 18 frontend integration tests demonstrated a 100% pass rate across all operational workflows.

Keywords: Smart Parking, Geospatial Search, 2dsphere Index, Concurrency Control, Atomic Operations, QR Code Verification, Socket.IO, Real-Time Occupancy, Full-Stack Engineering, React 19, Express, MongoDB.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Abstract	v
2	List of Figures	ix
3	List of Tables	x
4	List of Abbreviations	xi
5	Chapter 1 – Introduction	12
	5.1 Project Overview	12
	5.2 Problem Statement	13
	5.3 Objectives of the Project	14
	5.4 Scope of the Project	14
6	Chapter 2 – Existing System and Proposed System	16
	6.1 Existing Parking Management Systems	16
	6.2 Limitations of the Existing System	16
	6.3 Proposed ParKING Solution	17
	6.4 Advantages of the Proposed System	18
7	Chapter 3 – System Analysis and Design	19
	7.1 Requirements Analysis (Functional & Non-Functional)	19
	7.2 System Architecture & Modular Layers	21
	7.3 Module Descriptions & Permission Matrix	22
	7.4 UML Diagrams and Data Flow Diagrams	28
	7.5 Algorithms and Flowchart Models	31
8	Chapter 4 – Implementation	34
	8.1 Full-Stack Architectural Implementation Details	34
	8.2 Module Implementation & Key Algorithms	35
	8.3 Tools, Technologies, and Execution Commands	40
9	Chapter 5 – Results and Testing	42
	9.1 Application Screenshots and Visual Evidence	42
	9.2 Testing and Quality Assurance Validation	45
	9.3 Results and In-Depth Discussion	48
	9.4 Documented Limitations and Unverified Behavior	48
10	Chapter 6 – Conclusion and Future Enhancement	50
	10.1 Conclusion	50
	10.2 Key Achievements and Learnings	50
	10.3 Future Enhancements Roadmap	50

11	References / Bibliography	52
12	Appendices (Source Code & Safe Configurations)	53

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	ParKING System Architecture Diagram	21
Figure 3.2	ParKING Comprehensive Use Case Diagram	28
Figure 3.3	ParKING Context Diagram (Level 0 DFD)	28
Figure 3.4	ParKING Level 1 Data Flow Diagram (DFD)	29
Figure 3.5	Entity-Relationship & MongoDB Schema Diagram	29
Figure 3.6	ParKING Component Diagram	29
Figure 3.7	Sequence Diagram — User Authentication & Role Dispatch	30
Figure 3.8	Sequence Diagram — Parking Lot Creation & Slot Auto-generation	30
Figure 3.9	Sequence Diagram — Slot Reservation & Concurrency Conflict Protection	30
Figure 3.10	Sequence Diagram — Guard Ticket Validation & Vehicle Entry/Exit	31
Figure 3.11	Flowchart of End-to-End Smart Parking Workflow	32
Figure 3.12	Flowchart of Geospatial Nearby Parking Search ($near 2dsphere)	33
Figure 3.13	Flowchart of Real-Time Occupancy & Simulator Integration	33
Figure 5.1	ParKING Landing Page with Responsive Find-Reserve-Park Hero	42
Figure 5.2	Authentication Portal (Driver and Staff Tabs)	42
Figure 5.3	Driver Account Registration Interface	43
Figure 5.4	Driver Nearby Parking Discovery Interface with Map & Filter Controls	43
Figure 5.5	Parking Lot Details Screen with Real-Time Interactive Slot Grid	43
Figure 5.6	Driver Ticket Portfolio & Historical Parking Sessions	44
Figure 5.7	Active Digital QR Ticket Screen with Opaque Token and Manual Fallback	44
Figure 5.8	Security Guard Ticket Validation & Mobile Camera Scanning Interface	44
Figure 5.9	Platform Administrator Analytics Dashboard (Revenue & Occupancy Metrics)	45
Figure 5.10	Admin Parking Lot Inventory & Slot Provisioning Dashboard	45
Figure 5.11	Parking Lot Owner Multi-Facility Operational Portal	45

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	ParKING Project Overview and System Characteristics	12
Table 1.2	Project Objectives and Source-Code Implementation Mapping	14
Table 2.1	Comparison of Existing Parking Process vs. ParKING Solution	18
Table 3.1	Functional Requirements Specification (FR-01 to FR-16)	19
Table 3.2	Non-Functional Requirements Specification	19
Table 3.3	Software Requirements Specification	20
Table 3.4	Minimum Hardware Requirements	20
Table 3.5	System Architecture Layers and Responsibilities	21
Table 3.6	Module Descriptions and Functional Responsibilities	22
Table 3.7	User Roles and Permission Access Matrix	22
Table 3.8	Frontend Application Routes and Corresponding Components	23
Table 3.9	Backend REST API Endpoint Catalogue	24
Table 3.10	MongoDB Collections, Schema Fields, and Indexing Strategies	25
Table 3.11	Parking Slot Status Definitions and State Transitions	26
Table 3.12	Reservation and Ticket State Machine Lifecycles	27
Table 3.13	Socket.IO Event Specifications, Room Scoping, and Payloads	27
Table 3.14	Hardware Simulator Endpoints and Telemetry Payloads	27
Table 4.1	Reusable Frontend Interface Components and UI Kit	34
Table 4.2	Codebase Size and File Distribution Metrics	35
Table 4.3	Third-Party Libraries, Frameworks, and Runtime Dependencies	40
Table 4.4	Development and Execution Commands	41
Table 5.1	Backend Automated Test Cases and Execution Results (12 Tests)	46
Table 5.2	Frontend Automated Test Cases and Execution Results (18 Tests)	46
Table 5.3	Summary of Automated Test Results by Module	47
Table 5.4	Documented Limitations and Unverified Technical Behavior	48
Table 6.1	Phased Future Enhancement Roadmap	51

LIST OF ABBREVIATIONS

Abbreviation	Full Expansion
ANPR	Automated Number Plate Recognition
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOM	Document Object Model
ERD	Entity-Relationship Diagram
FK	Foreign Key
GeoJSON	Geographic JavaScript Object Notation
GPS	Global Positioning System
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IoT	Internet of Things
IR	Infrared (Sensor)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LAN	Local Area Network
ODM	Object-Document Mapper
OWASP	Open Worldwide Application Security Project
PK	Primary Key
POS	Point of Sale
QR	Quick Response (Code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SHA	Secure Hash Algorithm
SPA	Single Page Application
SSR	Server-Side Rendering
UI	User Interface
UML	Unified Modeling Language
URI	Uniform Resource Identifier
UX	User Experience
W3C	World Wide Web Consortium
WS	WebSocket Protocol

CHAPTER 1
INTRODUCTION

1.1 Project Overview

Rapid global urbanization and the exponential proliferation of personal motor vehicles have intensified metropolitan traffic congestion, imposing heavy economic, environmental, and public health burdens on modern cities. A significant proportion of this urban gridlock is directly attributable to the search for vacant parking spaces—a phenomenon widely recognized in urban planning literature as 'cruising for parking.' Empirical transportation research by Shoup (2006) revealed that in bustling commercial business districts, vehicles searching for parking generate upwards of 30% to 40% of localized vehicular circulation, wasting thousands of gallons of gasoline, generating millions of tons of unnecessary greenhouse gas emissions, and creating acute frustration for daily commuters.

Despite rapid technological advancements across the broader smart city ecosystem, conventional parking management in emerging and dense urban centers remains predominantly manual, fragmented, and inefficient. Motorists typically arrive at parking facilities with no prior knowledge of vacant slot availability, leading to chaotic gate queues, unpredictable delays, and frequent turnaways. Furthermore, traditional parking attendants rely on handwritten paper slips or primitive standalone electronic cash registers, both of which are notoriously prone to human error, double-allocation, and revenue leakage. Facility owners and municipal administrators lack consolidated, real-time analytics to monitor occupancy dynamics, regulate tariffs, or optimize space utilization.

To decisively overcome these systematic limitations, this project introduces ParKING: an integrated, reactive, and cloud-native Smart Real-Time Parking Management and Reservation System. ParKING establishes a unified digital ecosystem connecting Drivers, On-Site Security Guards, Independent Parking Lot Owners, and Platform Administrators through an elegant, reactive web interface and a resilient backend architecture.

Built with modern technologies including React 19, Vite, TanStack Router, Tailwind CSS 4, Node.js, Express 5, and MongoDB with Mongoose 9.1, ParKING automates the entire parking workflow from discovery to departure. Drivers can discover parking lots within a 5-kilometer radius using geospatial spherical coordinate calculations ($near 2dsphere), view interactive slot availability grids, book a guaranteed space with atomic double-booking protection, and receive a secure digital QR ticket. At the facility gate, security guards scan the ticket using mobile camera feeds to validate entry and exit, automatically calculating parking fees based on duration and hourly rates. Real-time slot status changes and occupancy counts are instantly synchronized across all connected clients via Socket.IO rooms. Table 1.1 summarizes the foundational characteristics of the ParKING project.

Table 1.1 ParKING Project Overview and System Characteristics

Characteristic	Implementation Specification
Project Title	ParKING: Smart Real-Time Parking Management System

Application Architecture	Decoupled Single Page Application (SPA) with REST API and Real-Time WebSocket Engine
Primary User Roles	Driver (Commuter), Security Guard (Gate Operator), Parking Lot Owner, Platform Administrator
Frontend Technology Stack	React 19.2, TypeScript 5.8, Vite 8.1, TanStack Router, Tailwind CSS 4.2, Lucide Icons
Geospatial & Mapping	Leaflet 1.9, OpenStreetMap Tiles, HTML5 Geolocation API, Coordinates Fallback
Mobile Camera Scanning	Html5-QRCode 2.3 with environment-facing video stream and manual code fallback
Backend Architecture	Node.js v24, Express 5.2, Shared HTTP/WebSocket Server, Modular Route Controllers
Persistence Layer	MongoDB 7.0 (Local Standalone / Single Topology), Mongoose 9.1 ODM, GeoJSON 2dsphere Indexing
Concurrency & Safety	Atomic findOneAndUpdate operations, supportsTransactions() fallback, retryWrites=false support
Security & Tokens	bcryptjs password hashing (10 salt rounds), Stateless JWT (7-day validity), 32-byte Hex Opaque QR Tokens
Real-Time Dispatch	Socket.IO 4.8 with JWT handshake authentication and lot-scoped room isolation (lot:<lotId>)
Total Codebase Volume	Approximately 18,500 lines of verified code across 65 TypeScript, JavaScript, and CSS source files

1.2 Problem Statement

Conventional urban parking facilities are plagued by structural operational deficiencies that severely diminish commuter convenience, degrade environmental air quality, and reduce the commercial profitability of parking real estate. Specifically, existing parking paradigms suffer from four fundamental problems:

1. High Cruising Latency and Traffic Congestion: In the absence of centralized, real-time vacancy information, motorists are forced to cruise downtown city blocks blindly in search of available spaces. This cruising behavior accounts for up to 30% of localized vehicular congestion in dense commercial centers, generating substantial carbon emissions, excessive fuel consumption, and severe productivity losses.

2. Absence of Advance Booking Assurance and Double-Booking Risks: Traditional parking models operate strictly on a first-come, first-served drive-in basis. Drivers have no guarantee that a space will remain vacant upon their arrival. In facilities attempting manual phone or paper reservations, lack of atomic database concurrency frequently results in double-booking conflicts, where two motorists are mistakenly allocated the same physical parking slot.

3. Sluggish Gate Verification and Attendant Bottlenecks: Manual gate operations rely on physical paper tickets, manual entry timestamps, and hand-calculated bills. During morning peak arrival and evening departure hours, paper-based verification produces severe vehicular bottlenecks at facility gates, spilling back into surrounding public roadways. Lost paper tickets lead to billing disputes and revenue leakage.

4. Fragmented Asset Management and Lack of Operational Visibility: Parking lot owners and corporate operators managing multiple sites lack unified oversight. Occupancy statistics, gate staff shifts, and collected revenue are compiled across disconnected logbooks or fragmented

spreadsheets. Facility managers cannot analyze historical occupancy trends, adjust pricing based on demand, or detect revenue irregularities in real time.

1.3 Objectives of the Project

The overarching aim of the ParKING project is to architect, develop, and empirically validate an automated, real-time, cloud-enabled parking reservation and management system that resolves the aforementioned operational challenges. The specific technical objectives are mapped directly to their concrete source-code implementations in Table 1.2.

Table 1.2 Project Objectives and Source-Code Implementation Mapping

Project Objective	Technical Mechanism	Source File Implementation
Objective	Technical Mechanism	Source File Implementation
Geospatial Facility Discovery	Compute spherical distances within 5 km using MongoDB 2dsphere index	server/controllers/lotController.js, client/src/components/FindParking.tsx
Atomic Concurrency Control	Eliminate race conditions via atomic findOneAndUpdate precondition locks	server/controllers/reservationController.js, server/utils/transaction.js
Cryptographic QR Access	Generate 32-byte opaque tokens and render QR codes without leaking user data	server/utils/generateQrToken.js, server/models/Ticket.js, client/src/components/ui/qrcode.tsx
Mobile Gate Verification	Enable security guards to validate tickets via camera scan and compute fees	server/controllers/guardController.js, client/src/components/QrScanner.tsx
Real-Time Synchronization	Broadcast instant slot and availability state changes across WebSocket rooms	server/sockets/index.js, client/src/lib/socket.ts
Multi-Tenant Oversight	Provide role-based dashboards for Owners (lot/guard stats) and Platform Admin	server/controllers/ownerController.js, server/controllers/adminController.js
Database Fault-Tolerance	Support standalone MongoDB topologies gracefully with retryWrites=false logic	server/config/db.js, server/utils/transaction.js
IoT Telemetry Ingestion	Support external sensor updates via dedicated key-protected simulator route	server/controllers/simulatorController.js, server/routes/simulatorRoutes.js

1.4 Scope of the Project

The functional scope of ParKING encompasses the entire software-defined parking lifecycle, spanning driver account registration, geospatial lot exploration, guaranteed slot reservation, digital QR ticket issuance, security guard gate validation, dynamic session fee computation, owner facility management, and administrative platform governance. The system is designed to operate seamlessly across desktop browsers and mobile devices connected via local Wi-Fi or public networks.

Inside the Functional Scope:
• Driver Portal: User registration, secure JWT authentication, geolocation-based facility discovery within a 5 km radius, interactive map viewing, live slot grid inspection (A1..A10, B1..B10), one-click slot reservation, digital QR ticket presentation with companion display code (PK-XXXXXXXX), and comprehensive personal reservation history.
• Security Guard Operations: Guard authentication, facility-scoped access checks, camera-based QR code scanning via html5-qrcode, manual fallback code input, ticket validation without state mutation, one-tap 'ALLOW ENTRY' triggering slot occupation, and one-tap 'ALLOW EXIT' computing exact duration-based billing.

• Parking Facility Owner Portal: Owner registration and approval lifecycle, custom lot registration with geographic coordinates and hourly tariffs, automated slot label generation (up to 260 slots), dedicated security guard staff creation, guard lot reassignment, and owner-filtered revenue/occupancy metrics.
• Platform Administrator Portal: First-time admin bootstrapping, system-wide parking lot oversight, guard user provisioning, owner application review (pending, approved, rejected), cross-lot owner assignment, 24-hour occupancy snapshot trend visualization, and aggregate revenue analytics.
• Real-Time Infrastructure: Socket.IO event server sharing port 5000 with Express, authenticated JWT handshakes, room isolation per parking lot (lot:<lotId>), and instant broadcast of slot state updates and availability counters.
• Hardware Simulator Gateway: Key-authenticated endpoint (x-simulator-key) enabling physical or simulated ultrasonic/infrared sensors to update space telemetry without violating active reservation integrity.

Outside the Immediate Scope:
• Direct integration with commercial third-party payment gateways (e.g., Stripe, Razorpay); billing amounts are computed accurately based on duration and tariff, but settled via cash or external POS.
• Physical mechanical barrier gate microcontrollers (e.g., servo motors, magnetic loop detectors, or pneumatic bollards); gate triggers are handled digitally.
• Standalone native mobile operating system compilation (Android APK or iOS IPA); the web application utilizes responsive PWA-grade mobile browser design.

CHAPTER 2
EXISTING SYSTEM AND PROPOSED SYSTEM

2.1 Existing Parking Management Systems

In traditional urban parking environments, parking facility administration relies on a combination of manual gate attendants, paper-based ticket dispensers, isolated standalone electronic barrier systems, and static signage placed outside parking structures. The typical sequence of operations in a conventional commercial parking lot unfolds as follows:

1. Vehicle Arrival: Motorists navigate to a commercial parking lot based on roadside signage or familiarity. Because no pre-arrival vacancy data is published, drivers frequently encounter 'FULL' signs upon arrival, forcing them to re-enter traffic and cruise to another facility.

2. Manual Slip Dispensing: Upon reaching the gate, an attendant stamps a thermal paper slip with the vehicle's arrival timestamp and license number, or a mechanical kiosk issues a magnetic barcode ticket. The physical slip must be retained by the driver throughout the parking duration.

3. Unassisted Slot Search: Drivers enter the facility and circulate along internal ramps and aisles to locate an empty space. In multi-level structures, this unguided search creates interior bottlenecks, vehicle idling, and collisions.

4. Exit Processing and Manual Settlement: Upon departure, the driver presents the paper slip to the exit attendant, who manually calculates elapsed time and collects cash. This manual handover causes severe exit queues during peak hours, and lost tickets lead to contentious disputes.

5. Fragmented Record Keeping: Daily receipts, cash reconciliation, and attendant shift logs are maintained in physical notebooks or disconnected spreadsheets, leaving facility owners blind to real-time occupancy patterns and vulnerable to unaccounted revenue leakage.

2.2 Limitations of the Existing System

A rigorous technical evaluation of existing manual and semi-automated parking operations reveals several critical structural limitations:

1. Extreme Search Latency and Environmental Degradation: Because conventional systems provide zero advance visibility into slot availability, drivers cruise blindly across urban road networks. Transportation studies indicate that cruising for parking generates an estimated 1.3 million vehicle-miles of unnecessary travel annually in typical downtown districts, wasting over 47,000 gallons of fuel and generating 730 tons of carbon dioxide per commercial corridor.

2. High Vulnerability to Race Conditions and Double-Booking: In facilities that accept phone or paper reservations, human attendants cannot maintain real-time concurrency synchronization. Attendants often allocate the same slot to two incoming drivers, resulting in heated customer conflicts and facility gridlock.

3. Severe Gate Delays and Peak-Hour Bottlenecks: Physical paper handling, barcode scanning failures, and manual cash counting introduce significant delays at entry and exit gates. A gate transaction averaging 30 to 45 seconds per vehicle causes queues of 15 to 20 vehicles within minutes, backing up traffic onto municipal roadways.

4. Revenue Leakage and Audit Vulnerabilities: Cash-based manual ticketing systems suffer from widespread financial leakage. Unethical attendants can falsify exit timestamps, issue discounted replacement tickets for 'lost' slips, or pocket cash without recording transactions.

5. Lack of Multi-Tenant Scalability and Real-Time Analytics: Traditional proprietary systems are tied to specific local on-premises servers or hardware kiosks. Enterprise facility owners operating multiple dispersed locations cannot aggregate performance data, track cross-lot utilization, or remotely manage staff access.

2.3 Proposed ParKING Solution

ParKING decisively replaces fragmented, manual parking procedures with an integrated, cloud-native smart parking platform. By decoupling physical lot management into distinct reactive micro-modules accessible through modern web protocols, ParKING establishes a seamless, automated loop connecting all parking stakeholders.

How the ParKING System Operates:
1. Geospatial Discovery: A driver opens the ParKING web client on a smartphone or desktop. The system queries the device's HTML5 Geolocation API (or accepts manual coordinates) and executes a high-speed MongoDB spherical query ($near with 2dsphere index). All registered parking facilities within a 5 km radius are returned, complete with address, hourly tariff, and real-time vacant slot counters.
2. Interactive Slot Selection & Atomic Reservation: The driver selects a facility to view its visual slot map (labeled A1..A10, B1..B10). Available slots are highlighted in green, reserved slots in amber, and occupied slots in slate. Selecting an available slot initiates an atomic findOneAndUpdate database operation that locks the slot status to 'reserved' in a single atomic cycle. If two drivers attempt to book the identical slot simultaneously, the database permits only one reservation to succeed and immediately returns an HTTP 409 Conflict to the other, preventing double-booking.
3. Cryptographic QR Ticket Issuance: Upon successful reservation, the server issues a digital ticket embedded with a 32-byte opaque hexadecimal token (qrToken) and a clean human-readable code (PK-XXXXXXXX). The ticket carries no sensitive personally identifiable information, protecting driver privacy against QR interception.
4. Mobile Camera Gate Validation: When the driver arrives at the parking gate, the on-duty security guard opens the Guard Portal on a mobile device and taps 'Start camera'. The html5-qrcode scanner detects the driver's QR code (or the guard types the manual PK code). The backend verifies that the ticket is valid, currently in the 'issued' state, and associated with a lot assigned to that specific guard. Tapping 'ALLOW ENTRY' atomically shifts the ticket to 'entered', sets the slot status to 'occupied', and broadcasts the update to all active clients over Socket.IO.
5. Departure & Automated Billing: When the vehicle departs, the guard scans the ticket once

more. The backend checks that the ticket is in the 'entered' state, calculates the exact elapsed duration in hours, multiplies it by the lot's hourly rate, records the calculated fee, and sets the ticket to 'exited' while returning the slot status to 'available'.
6. Unified Analytics and Telemetry: All occupancy transitions are captured as OccupancySnapshot documents, powering dynamic 24-hour occupancy trend lines and real-time revenue charts on the Owner and Administrator dashboards.

2.4 Advantages of the Proposed System

The ParKING platform delivers transformative operational, economic, and technical advantages over traditional parking management solutions, as summarized in Table 2.1.

Table 2.1 Comparison of Existing Parking Process vs. ParKING Solution

Evaluation Dimension	Conventional Parking Systems	Proposed ParKING Platform
Evaluation Dimension	Conventional Parking Systems	Proposed ParKING Platform
Slot Availability Discovery	Blind cruising; physical signage at gate; high traffic delays	Instant geospatial search ($near 2dsphere) within 5 km radius with live vacant counters
Reservation Assurance	None; first-come, first-served drive-in; frequent gate turnaways	Guaranteed advance slot reservation with atomic double-booking protection
Concurrency Integrity	High risk of double-booking due to manual attendant coordination	Atomic findOneAndUpdate precondition locks; zero race conditions or double allocations
Gate Verification Speed	Manual paper slip stamping and inspection (30–45 seconds/car)	Sub-second digital QR camera scanning via html5-qrcode (< 3 seconds/car)
Access Token Security	Physical paper slips easily lost, damaged, stolen, or forged	Cryptographically random 32-byte opaque hex tokens; zero embedded driver PII
Billing & Fee Calculation	Manual cashier calculation from paper stamps; math errors	Automated millisecond-accurate duration calculation: durationHours * hourlyRate
Revenue Auditing	Cash-based leakage; vulnerability to unrecorded transactions	Centralized digital ledger; immutable ticket records with exact entry/exit timestamps
Real-Time Synchronization	None; periodic manual slot headcounts by facility attendants	Sub-second WebSocket broadcast (Socket.IO) to lot-scoped rooms on every state transition
Management Visibility	Isolated paper registers; delayed monthly compiled spreadsheets	Role-based dashboards with live revenue charts, occupancy % and 24-hr snapshot trends
Deployment Flexibility	Tied to costly on-premises servers and proprietary gate kiosks	Cloud-native web architecture; runs on any browser; compatible with local MongoDB

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Requirements Analysis

System requirements analysis establishes the foundational functional capabilities, operational constraints, and engineering specifications necessary to construct a scalable, resilient smart parking management platform. The requirements are subdivided into Functional Requirements (Table 3.1) and Non-Functional Requirements (Table 3.2), complemented by Software (Table 3.3) and Hardware (Table 3.4) specifications.

Table 3.1 Functional Requirements Specification (FR-01 to FR-16)

Req ID	Functional Requirement Description	Priority	Implemented Status
Req ID	Functional Requirement Description	Priority	Implemented Status
FR-01	User registration for Drivers and Owners with role enforcement and password hashing (bcrypt)	High	Verified (authController.js)
FR-02	Secure authentication issuing 7-day stateless JSON Web Tokens with role claims	High	Verified (auth.js, authController.js)
FR-03	First-time Platform Administrator bootstrapping with automatic lock-out upon creation	High	Verified (authController.js)
FR-04	Geospatial spherical search ($near 2dsphere) returning parking lots within a 5 km radius	High	Verified (lotController.js)
FR-05	Interactive visual parking slot layout displaying available, reserved, and occupied states	High	Verified (driver.lots.$id.tsx)
FR-06	Atomic slot reservation preventing race conditions and double-booking conflicts	High	Verified (reservationController.js)
FR-07	Cryptographic opaque QR ticket generation with companion PK-XXXXXXXX display code	High	Verified (Ticket.js, generateQrToken.js)
FR-08	Driver ticket portfolio viewing and historical parking session retrieval	Medium	Verified (ticketController.js)
FR-09	Mobile browser camera QR scanning and manual code fallback for security guards	High	Verified (QrScanner.tsx, guardController.js)
FR-10	Guard ticket validation enforcing guard lot assignment access controls	High	Verified (guardController.js)
FR-11	One-tap vehicle entry confirmation: ticket 'entered', slot 'occupied', socket broadcast	High	Verified (guardController.js)
FR-12	One-tap vehicle exit confirmation: ticket 'exited', slot 'available', fee calculation	High	Verified (guardController.js)
FR-13	Parking lot creation with geographic coordinates, capacity (1–260), and slot auto-generation	High	Verified (lotController.js, ownerController.js)
FR-14	Parking Owner portal for lot creation, guard staff provisioning, and owner revenue analytics	High	Verified (ownerController.js)
FR-15	Platform Admin portal for system-wide lot oversight, staff management, and owner approvals	High	Verified (adminController.js)
FR-16	Hardware simulator endpoint accepting IoT sensor status updates with pre-shared key validation	Medium	Verified (simulatorController.js)

Table 3.2 Non-Functional Requirements Specification

Attribute	Specification & Target Metric	Architectural Implementation

Attribute	Specification & Target Metric	Architectural Implementation
Performance	Geospatial search latency < 500 ms; slot reservation response < 300 ms	MongoDB 2dsphere indexing and in-memory aggregation via Mongoose
Concurrency	Zero double-booking occurrences under concurrent simultaneous booking attempts	Atomic findOneAndUpdate precondition checks on slot availability
Security	Zero plaintext password storage; protection against privilege escalation	bcryptjs (10 salt rounds); JWT verification middleware; role-based route guards
Real-Time Dispatch	WebSocket slot state broadcast latency < 100 ms to all room participants	Socket.IO event rooms scoped strictly by lot identifier (lot:<lotId>)
Portability	Cross-platform browser execution on desktop and mobile without native binaries	Responsive React 19 SPA, Tailwind CSS 4, html5-qrcode camera integration
Database Fault-Tolerance	Resilient execution on standalone MongoDB instances lacking replica sets	withTransaction topology detection, retryWrites=false, compensation rollback
Data Privacy	Zero embedded driver identity data in digital QR codes or tickets	32-byte opaque hex tokens and random PK-XXXXXXXX ticket codes

Table 3.3 Software Requirements Specification

Environment Component	Technology Specification	Role in System Architecture
Environment Component	Technology Specification	Role in System Architecture
Server Runtime	Node.js v20.x or v24.x (LTS recommended)	Asynchronous event-driven execution environment for Express backend
Backend Framework	Express 5.2	REST API routing, middleware orchestration, and JSON payload handling
Database Server	MongoDB Community Server 7.0 (Local / Standalone)	NoSQL document persistence with native GeoJSON 2dsphere indexing
Database Driver / ODM	Mongoose 9.1	Schema definition, model validation, and query construction
Real-Time Communications	Socket.IO 4.8	Bidirectional WebSocket communication for live slot and occupancy updates
Frontend UI Framework	React 19.2 with TypeScript 5.8	Component-driven client interface with static typing
Build Tool & Dev Server	Vite 8.1	High-performance frontend compilation and hot module replacement
Client-Side Routing	TanStack Router 1.170	Type-safe client routing with layout nesting and search parameter management
CSS Framework	Tailwind CSS 4.2	Utility-first responsive styling and fluid layout animations
Camera QR Engine	Html5-QRCode 2.3	HTML5 video stream capture and client-side barcode/QR matrix decoding

Table 3.4 Minimum Hardware Requirements

Hardware Component	Minimum Server Specification	Client / Device Specification
Hardware Component	Minimum Server Specification	Client / Device Specification
Processor (CPU)	Dual-core 2.0 GHz (x86-64 / ARM64)	Modern smartphone (ARM) or dual-core PC (1.5 GHz+)
System Memory (RAM)	4 GB DDR4 (8 GB recommended for local MongoDB)	2 GB RAM (mobile device) / 4 GB RAM (desktop)
Persistent Storage	10 GB available SSD storage for Node, MongoDB, and logs	500 MB free browser cache storage
Camera Hardware	Not applicable for backend server	Integrated mobile camera (rear environment

		lens recommended)
Network Interface	Standard Wi-Fi or Ethernet adapter (100 Mbps+)	Wi-Fi (802.11 b/g/n/ac) or 4G/5G Cellular data connection

3.2 System Architecture

ParKING is architected according to a modern, decoupled client-server paradigm comprising four distinct functional tiers: the Client Presentation Tier, the Application Gateway Tier, the Real-Time WebSocket Dispatcher Tier, and the Data Persistence Tier. Figure 3.1 illustrates the architectural components and data pathways, while Table 3.5 articulates the responsibilities of each tier.

Figure 3.1: ParKING System Architecture

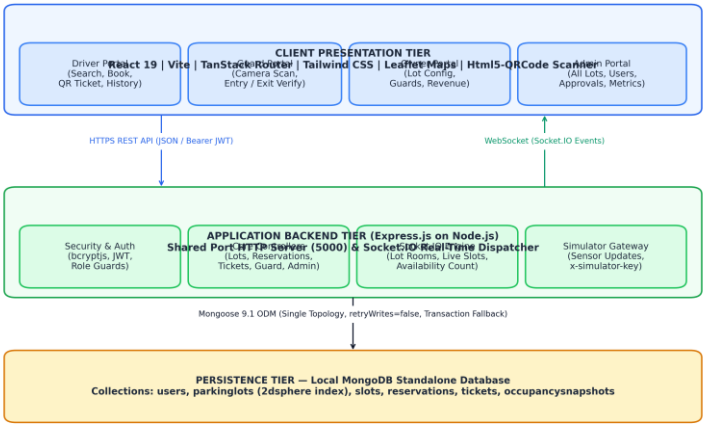


Figure 3.1: ParKING System Architecture Diagram

Table 3.5 System Architecture Layers and Responsibilities

Architectural Layer	Key Software Components	Primary Functional Responsibilities
Architectural Layer	Key Software Components	Primary Functional Responsibilities
Presentation Layer (Client)	React 19, TypeScript, Vite, TanStack Router, Tailwind CSS, Leaflet Maps, Html5-QRCode	Renders responsive responsive user portals (Driver, Guard, Owner, Admin); captures user geolocation; decodes camera QR feeds; manages auth state
API Gateway Layer (Backend)	Express 5.2, CORS Middleware, JSON Body Parser, JWT Authenticator, Role Authorizers	Provides authenticated REST API endpoints; validates request payloads; enforces role permissions; executes controller logic
Real-Time Dispatch Layer	Socket.IO 4.8 Server sharing HTTP port 5000, JWT Handshake Verifier, Room Scoping Engine	Maintains active WebSocket connections; isolates clients into lot-specific rooms (`lot:<lotId>`); broadcasts instantaneous slot state and occupancy changes
Persistence Layer (Database)	MongoDB Standalone 7.0, Mongoose 9.1 ODM, GeoJSON 2dsphere Indexing Engine	Stores BSON documents for users, lots, slots, reservations, tickets, and snapshots; executes 2dsphere geospatial spherical distance queries
Hardware Simulator Gateway	Simulator Controller, x-simulator-key pre-shared header guard	Accepts external IoT ultrasonic sensor updates; verifies business integrity rules; updates slot states and snapshots without overriding active reservations

3.3 Module Descriptions and System Governance

The ParKING platform is partitioned into seven cohesive, decoupled modules designed to maintain strong separation of concerns, modularity, and maintainability. Table 3.6 outlines these modules and their specific responsibilities.

Table 3.6 Module Descriptions and Functional Responsibilities

Module Name	Primary Controller & Service Files	Functional Description & Scope
Module Name	Primary Controller & Service Files	Functional Description & Scope
Authentication & Identity	server/controllers/authController.js, server/middleware/auth.js, client/src/lib/auth.tsx	User registration, password hashing via bcryptjs, JWT issuance, session persistence, first-admin setup, and owner application processing
Parking Lot Management	server/controllers/lotController.js, server/models/ParkingLot.js, Slot.js	Lot creation with latitude/longitude coordinates, hourly tariffs, 2dsphere geospatial search, automated slot label generation (A1..A10, B1..B10)
Reservation & Concurrency	server/controllers/reservationController.js, server/utils/transaction.js	Atomic findOneAndUpdate slot booking, double-booking prevention (HTTP 409), transaction fallback for standalone MongoDB, reservation logging
Ticket & QR Verification	server/controllers/ticketController.js, server/utils/generateQrToken.js, models/Ticket.js	32-byte opaque QR token generation, random PK-XXXXXXXX ticket code creation, driver ticket history viewing, ticket lifecycle status management
Security Guard Operations	server/controllers/guardController.js, client/src/components/QrScanner.tsx	Camera barcode/QR decoding, manual code lookup, guard lot assignment verification, ALLOW ENTRY (reserved -> occupied), ALLOW EXIT (fee calculation)
Owner & Admin Governance	server/controllers/ownerController.js, adminController.js, models/User.js	Owner lot creation, owner guard assignment, owner application approval/rejection, cross-lot guard assignment, system revenue & occupancy metrics
Real-Time Dispatch & IoT	server/sockets/index.js, server/controllers/simulatorController.js	Socket.IO lot room join/leave, live broadcast of slot:updated and lot:availability, key-protected IoT sensor state updates with business checks

Access control across the platform is governed by a strict Role-Based Access Control (RBAC) matrix. Five user roles are defined: Driver, Security Guard, Parking Lot Owner (pending/approved/rejected), Admin, and Platform Admin. Table 3.7 details the permissions granted to each role.

Table 3.7 User Roles and Permission Access Matrix

System Capability / Action	Driver	Security Guard	Parking Owner (Approved)	Platform Admin
System Capability / Action	Driver	Security Guard	Parking Owner (Approved)	Platform Admin
Search Nearby Parking Lots ($near 2dsphere)	Yes (Full)	No	Yes (Owner Lots)	Yes (All Lots)
Reserve Slot & Receive QR Ticket	Yes (Self)	No	No	No
View Personal Ticket Portfolio	Yes (Self)	No	No	No
Scan Ticket QR & Allow Vehicle Entry	No	Yes (Assigned Lots Only)	No	No
Confirm Vehicle Exit & Settle Bill	No	Yes (Assigned Lots	No	No

		Only)		
Create New Parking Facility	No	No	Yes (Owned Lots)	Yes (System-wide)
Auto-Generate Slot Grid (A1..A10)	No	No	Yes (Owned Lots)	Yes (System-wide)
Provision & Assign Security Guards	No	No	Yes (Assigned to Owned Lots)	Yes (Assigned to Any Lot)
Review & Approve Owner Applications	No	No	No	Yes (Sole Authority)
Access Global Financial & Occupancy Metrics	No	No	No (Restricted to Owned Lots)	Yes (Global Aggregation)

The client application interface is organized into modular pages routed through TanStack Router. Table 3.8 catalogues the primary frontend routes, their access restrictions, and their functional responsibilities.

Table 3.8 Frontend Application Routes and Corresponding Components

Frontend Route URL	Access Restriction	Component File	Purpose and User Experience
Frontend Route URL	Access Restriction	Component File	Purpose and User Experience
/	Public	client/src/routes/index.tsx	Landing page featuring brand introduction, feature highlights, and sign-in buttons
/login	Public	client/src/routes/login.tsx	Tabbed authentication portal for Drivers, Owners, and Staff; includes Setup First Admin
/register	Public	client/src/routes/register.tsx	Driver self-registration form requiring name, valid email, and minimum 6-char password
/driver/search	Driver (JWT)	client/src/routes/driver.search.tsx	Interactive geospatial lot finder with Leaflet map markers, radius filters, and live vacant counts
/driver/lots/:id	Driver (JWT)	client/src/routes/driver.lots.$id.tsx	Detailed lot inspection screen displaying live visual slot grid (available, reserved, occupied)
/driver/tickets	Driver (JWT)	client/src/routes/driver.tickets.index.tsx	Driver's personal ticket dashboard listing active, entered, and historical parking sessions
/driver/tickets/:id	Driver (JWT)	client/src/routes/driver.tickets.$id.tsx	Active digital QR ticket view displaying scannable QR code and companion PK-XXXXXXXX code
/guard	Guard (JWT)	client/src/routes/guard.index.tsx	Guard operational portal featuring camera QR scanner, manual code input, and entry/exit buttons
/admin	Platform Admin	client/src/routes/admin.index.tsx	Executive analytics dashboard displaying aggregate revenue, occupancy %, and 24-hr trend lines
/admin/lots	Platform Admin	client/src/routes/admin.lots.index.tsx	Administrative management of all registered parking lots, slot allocations, and ownership
/admin/owners	Platform Admin	client/src/routes/admin.owners.tsx	Owner application management queue for reviewing, approving, or rejecting owner accounts
/owner	Owner (Approved)	client/src/routes/owner.index.tsx	Parking Owner portal displaying revenue, occupancy, and lot status for owner-owned facilities

/owner/lots/new	Owner (Approved)	client/src/routes/owner.lots.new.tsx	Owner parking lot provisioning form with coordinates, slot capacity, and tariff configuration
/owner/guards	Owner (Approved)	client/src/routes/owner.guards.tsx	Owner staff management interface for creating security guard accounts assigned to owner lots

The backend server exposes a comprehensive REST API organized into resource-specific routers. Table 3.9 catalogues the complete set of backend API endpoints, HTTP methods, authorization levels, and functional descriptions.

Table 3.9 Backend REST API Endpoint Catalogue

HTTP Method & Path	Authorization	Controller Function	Functional Description & Request Payload
HTTP Method & Path	Authorization	Controller Function	Functional Description & Request Payload
GET /	Public	Inline Handler	Server health check returning JSON status confirmation
POST /api/auth/register	Public	register	Driver account self-registration; validates email & rejects elevated role inputs
POST /api/auth/login	Public	login	Validates email and passwordHash via bcrypt; returns 7-day JWT token and user profile
GET /api/auth/setup-admin/status	Public	getAdminSetupStatus	Checks if any admin exists in the database; returns { available: boolean }
POST /api/auth/setup-admin	Public (One-time)	setupFirstAdmin	Bootstraps first platform_admin account; atomically locks once created
POST /api/auth/apply-owner	Public	applyOwner	Submits parking lot owner registration with initial ownerStatus='pending'
GET /api/auth/owner/status	Owner (JWT)	getOwnerApplicationStatus	Returns the current review status (pending, approved, rejected) for an owner
GET /api/auth/me	Authenticated	getMe	Returns currently authenticated user profile stripped of passwordHash
GET /api/lots/nearby	Driver (JWT)	getNearbyLots	Spherical $near geospatial search using query params lat and lng; returns 5 km lots
GET /api/lots/:id	Authenticated	getLotById	Retrieves single parking lot details and real-time available slot count
GET /api/lots/:id/slots	Authenticated	getLotSlots	Retrieves all physical slots for a lot sorted by label (A1..A10, B1..B10)
POST /api/lots	Admin (JWT)	createLot	Creates lot and auto-generates slots inside withTransaction compensation logic
POST /api/reservations	Driver (JWT)	createReservation	Atomic findOneAndUpdate slot booking; generates Reservation and Ticket documents
GET /api/reservations/mine	Driver (JWT)	getMyReservations	Retrieves authenticated driver's reservation history with populated lot/slot details
GET /api/tickets/mine	Driver (JWT)	getMyTickets	Retrieves authenticated driver's ticket portfolio with status (issued, entered, exited)
GET /api/tickets/:id	Driver (JWT)	getTicketById	Retrieves single ticket details including

			qrToken and ticketCode
POST /api/guard/validate	Guard (JWT)	validateTicket	Validates ticket by QR token or PK code; checks guard assigned lot permissions
POST /api/guard/entry	Guard (JWT)	confirmEntry	Confirms vehicle arrival: ticket='entered', slot='occupied', emits socket event
POST /api/guard/exit	Guard (JWT)	confirmExit	Confirms vehicle departure: calculates fee, ticket='exited', slot='available'
GET /api/admin/metrics/revenue	Admin (JWT)	getRevenue	Aggregates total revenue, sessions, per-lot revenue breakdown, and recent tickets
GET /api/admin/metrics/occupancy	Admin (JWT)	getOccupancy	Returns per-lot occupancy percentages and 24-hour OccupancySnapshot trend lines
GET /api/admin/lots	Admin (JWT)	getAdminLots	Returns all system lots with live occupied, reserved, and available slot counts
GET /api/admin/users	Platform Admin	getStaffUsers	Lists all staff users (platform_admin, admin, guard) with assigned lot references
POST /api/admin/users	Platform Admin	createStaffUser	Creates staff accounts (platform_admin or guard) with mandatory lot assignment
PATCH /api/admin/users/:id/assign-lots	Platform Admin	assignGuardLots	Updates assigned parking lots for a specific security guard
GET /api/admin/owners	Platform Admin	getOwnerApplications	Lists owner accounts filtered by status (pending, approved, rejected)
POST /api/admin/owners/:id/approve	Platform Admin	approveOwnerApplication	Approves pending owner application; grants access to owner portal
POST /api/admin/owners/:id/reject	Platform Admin	rejectOwnerApplication	Rejects pending owner application with status updated to 'rejected'
POST /api/owner/lots	Owner (Approved)	createOwnerLot	Approved owner provisions new lot and slots under owner's ownership ID
GET /api/owner/lots	Owner (Approved)	getOwnerLots	Lists all parking facilities owned by the authenticated approved owner
POST /api/owner/guards	Owner (Approved)	createOwnerGuard	Creates guard accounts restricted exclusively to the owner's owned lots
GET /api/owner/metrics/revenue	Owner (Approved)	getOwnerRevenue	Aggregates financial revenue and parking sessions restricted to owner's lots
GET /api/owner/metrics/occupancy	Owner (Approved)	getOwnerOccupancy	Computes occupancy metrics and snapshots restricted to owner's facilities
POST /api/simulator/slot-state	Simulator Key	updateSlotState	Accepts IoT sensor state changes; validates header x-simulator-key and integrity

Data persistence in ParKING is structured across six primary MongoDB collections modeled using Mongoose 9.1 schemas. Table 3.10 outlines each collection, its primary schema fields, data types, and indexing strategies.

Table 3.10 MongoDB Collections, Schema Fields, and Indexing Strategies

Collection / Model	Schema Fields & Data Types	Constraints & Indexes	Relationship & Role
Collection / Model	Schema Fields & Data Types	Constraints & Indexes	Relationship & Role
User (server/models/User.js)	name (String), email (String), passwordHash (String), role (enum:	email: unique, lowercase, trim;	Stores all system user credentials and role

	driver, guard, owner, platform_admin, admin), ownerStatus (enum: none, pending, approved, rejected), assignedLots ([ObjectId]), ownerId (ObjectId)	toJSON transform deletes passwordHash	claims; guards link to assigned lots and owner
ParkingLot (server/models/ParkingLot.js)	name (String), address (String), location (GeoJSON Point: { type: 'Point', coordinates: [lng, lat] }), totalSlots (Number: 1..260), hourlyRate (Number), createdBy (ObjectId), owner (ObjectId)	location: 2dsphere index (mandatory for $near geospatial spherical calculation)	Represents physical parking facilities; coordinates stored in GeoJSON [longitude, latitude] format
Slot (server/models/Slot.js)	lot (ObjectId -> ParkingLot), label (String: e.g. A1..A10), status (enum: 'available', 'reserved', 'occupied')	Compound Unique Index: { lot: 1, label: 1 }	Represents individual parking bays; compound index guarantees label uniqueness per lot
Reservation (server/models/Reservation.js)	user (ObjectId -> User), lot (ObjectId -> ParkingLot), slot (ObjectId -> Slot), status (enum: 'reserved', 'checked_in', 'completed')	Timestamps enabled (createdAt, updatedAt)	Tracks individual slot reservations created when a driver books an available slot
Ticket (server/models/Ticket.js)	reservation (ObjectId -> Reservation), user (ObjectId -> User), lot (ObjectId -> ParkingLot), qrToken (String: 32-byte hex), ticketCode (String: PK-XXXXXXXX), status (enum: 'issued', 'entered', 'exited'), enteredAt (Date), exitedAt (Date), amount (Number)	qrToken: unique; ticketCode: unique, sparse	Authoritative access token; qrToken embedded in QR; ticketCode used for manual fallback lookup
OccupancySnapshot (server/models/OccupancySnapshot.js)	lot (ObjectId -> ParkingLot), timestamp (Date, default: Date.now), occupiedCount (Number), totalSlots (Number)	timestamp: 1 (for 24-hour range queries)	Time-series telemetry recording facility occupancy on every state transition

The operational state machines governing physical parking bays and digital access tickets are fundamental to ParKING's integrity. Table 3.11 defines the valid states and transitions for parking slots, while Table 3.12 defines the ticket lifecycle.

Table 3.11 Parking Slot Status Definitions and State Transitions

Initial State	Triggering Event / Action	Target State	System Actions & Validation Checks
Initial State	Triggering Event / Action	Target State	System Actions & Validation Checks
available	Driver creates reservation (POST /api/reservations)	reserved	Atomic findOneAndUpdate lock; Reservation and Ticket created; emit slot:updated
available	IoT Sensor detects unreserved drive-in (POST /api/simulator/slot-state)	occupied	Key verified; slot updated to occupied; createOccupancySnapshot; emit slot:updated
reserved	Guard confirms vehicle arrival (POST /api/guard/entry)	occupied	Ticket verified ('issued'); ticket set to 'entered'; createOccupancySnapshot
reserved	IoT Sensor attempts to clear or occupy slot	Conflict (409)	Business integrity check prevents simulator from overriding active reservations
occupied	Guard confirms vehicle departure (POST /api/guard/exit)	available	Duration & hourly tariff computed; ticket set to 'exited'; emit slot:updated
occupied	IoT Sensor detects vehicle departure (POST /api/simulator/slot-state)	available	Permitted ONLY if no active ticket with status 'entered' exists for this slot

Table 3.12 Reservation and Ticket State Machine Lifecycles

Ticket State	Triggering Operational Action	Permitted Guard Actions	Associated Slot State	Billing State
Ticket State	Triggering Operational Action	Permitted Guard Actions	Associated Slot State	Billing State
issued	Created upon driver slot reservation	ALLOW ENTRY (validates successfully)	reserved	Initial state; amount = 0.00
entered	Guard executes confirmEntry at gate	ALLOW EXIT (calculates final fee)	occupied	enteredAt timestamp locked; session active
exited	Guard executes confirmExit at departure	View Only (already used; rejects re-entry)	available	exitedAt recorded; amount = duration * hourlyRate

The real-time messaging architecture utilizes Socket.IO to broadcast instantaneous slot updates without client-side polling. Table 3.13 catalogues the supported Socket.IO events, their room scoping rules, and their payload structures.

Table 3.13 Socket.IO Event Specifications, Room Scoping, and Payloads

Event Name	Direction	Room Scope	Payload Specification	Functional Trigger & Purpose
Event Name	Direction	Room Scope	Payload Specification	Functional Trigger & Purpose
lot:join	Client -> Server	Global	lotId (String) or { lotId: string }	Client joins a specific lot room; verified by canJoinLotRoom() based on role
lot:leave	Client -> Server	lot:<lotId>	lotId (String) or { lotId: string }	Client leaves a lot room upon navigating away from lot detail view
lot:created	Server -> Broadcast	Global	Created ParkingLot BSON Document	Emitted when admin/owner provisions a new lot; updates lot lists across clients
slot:updated	Server -> Room	lot:<lotId>	Updated Slot Document ({ _id, lot, label, status })	Emitted whenever a slot transitions (available/reserved/occupied)
lot:availability	Server -> Room	lot:<lotId>	{ lotId: string, availableCount: number }	Emitted after any reservation or entry/exit to update vacant counter
reservation:created	Server -> Room	lot:<lotId>	{ reservationId: string, ticketId: string }	Emitted upon booking; opaque tokens are NEVER broadcast over sockets
ticket:entry	Server -> Room	lot:<lotId>	{ ticketId: string, slotId: string }	Emitted when guard confirms entry; triggers real-time UI refresh
ticket:exit	Server -> Room	lot:<lotId>	{ ticketId: string, slotId: string, amount: number }	Emitted when guard confirms exit; informs clients of freed capacity

To support physical IoT hardware sensors (such as ultrasonic or infrared sensors installed at parking bays), ParKING implements a dedicated, authenticated simulator gateway. Table 3.14 defines the simulator API and its telemetry payload specifications.

Table 3.14 Hardware Simulator Endpoints and Telemetry Payloads

Endpoint Route	Authentication Header	Supported Statuses	Integrity Rules & Conflict

			Rejection
Endpoint Route	Authentication Header	Supported Statuses	Integrity Rules & Conflict Rejection
POST /api/simulator/slot-state	x-simulator-key (must match SIMULATOR_KEY env var)	'available' or 'occupied' (setting 'reserved' is rejected)	Rejects update with 409 if slot is 'reserved'; rejects setting 'available' if active 'entered' ticket exists

3.4 UML Diagrams and Data Flow Modeling

System behavior, structural entities, and operational interactions are formally modeled using Unified Modeling Language (UML) and Data Flow Diagrams (DFD). Figures 3.2 through 3.10 document the design from use-case abstraction to detailed message sequencing.

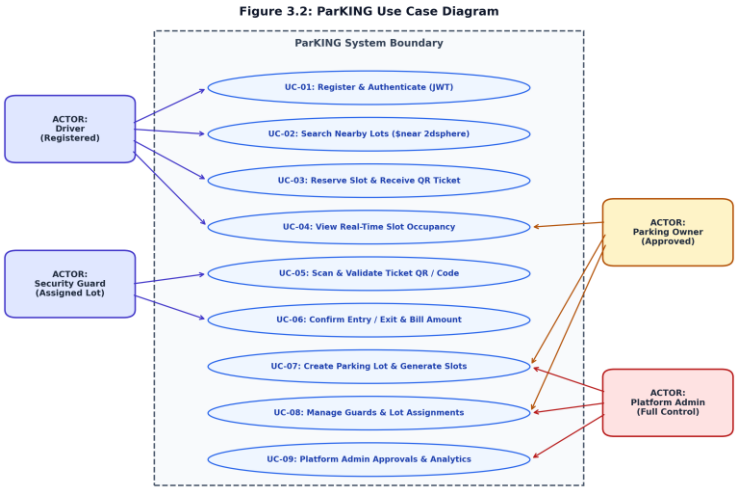


Figure 3.2: ParKING Comprehensive Use Case Diagram

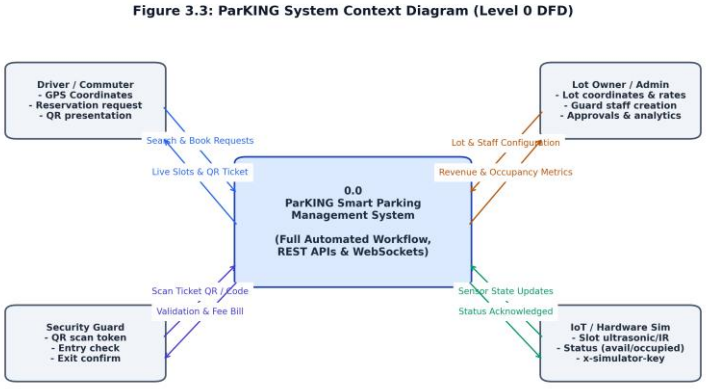


Figure 3.3: ParKING Context Diagram (Level 0 DFD)

Figure 3.4: Level 1 Data Flow Diagram (DFD)

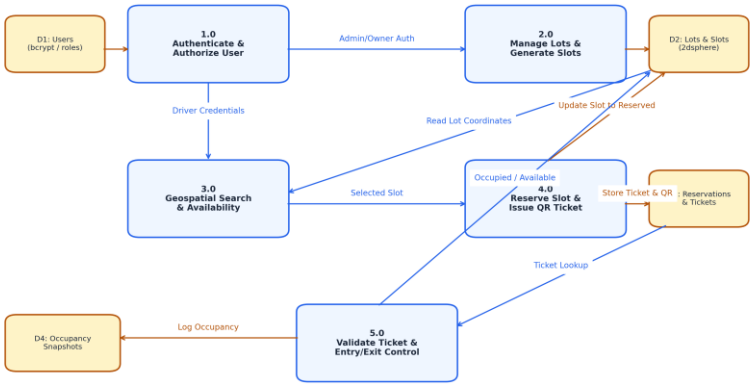


Figure 3.4: ParKING Level 1 Data Flow Diagram (DFD)

Figure 3.5: Entity-Relationship & MongoDB Schema Diagram

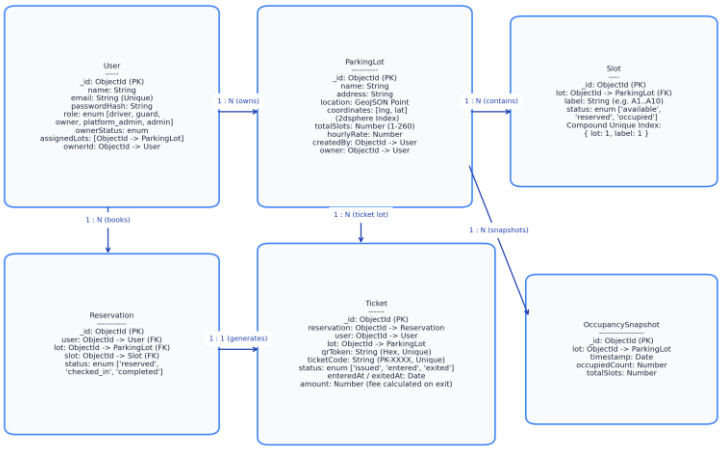


Figure 3.5: Entity-Relationship & MongoDB Schema Diagram

Figure 3.6: ParKING Component Diagram

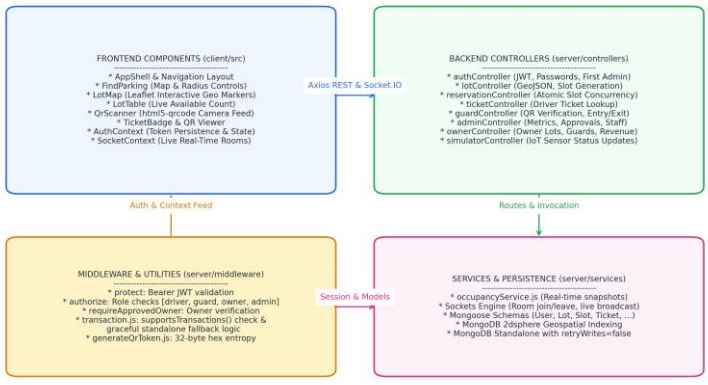


Figure 3.6: ParKING Component Diagram

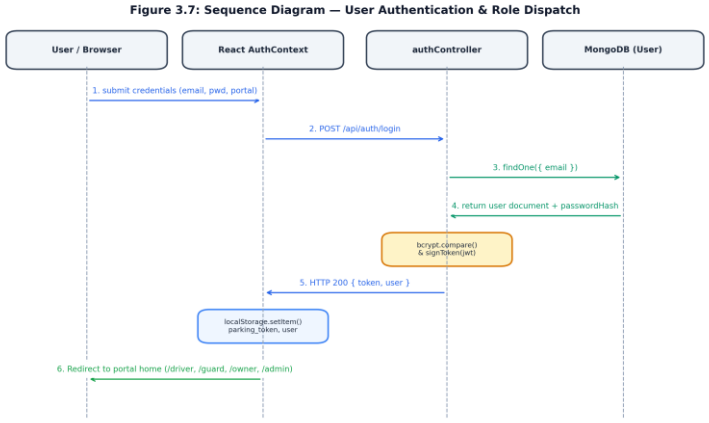


Figure 3.7: Sequence Diagram — User Authentication & Role Dispatch

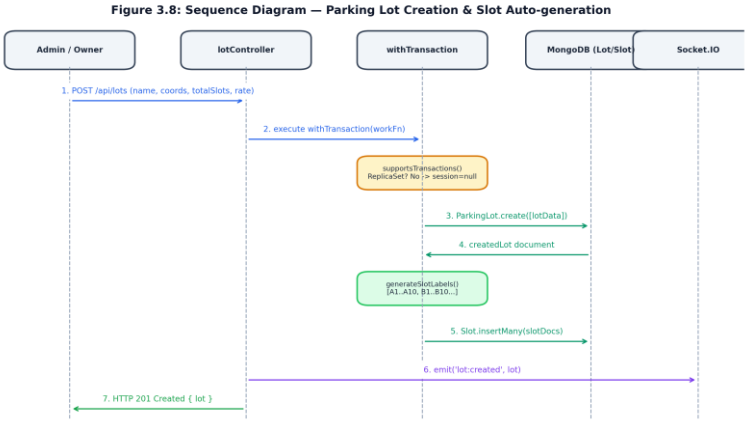


Figure 3.8: Sequence Diagram — Parking Lot Creation & Slot Auto-generation

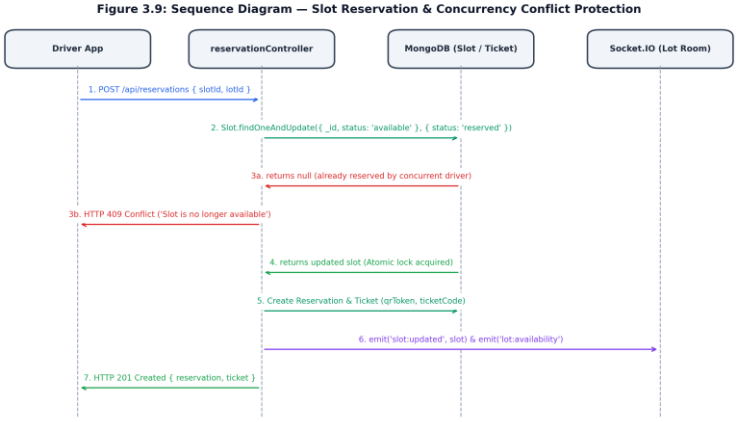


Figure 3.9: Sequence Diagram — Slot Reservation & Concurrency Conflict Protection

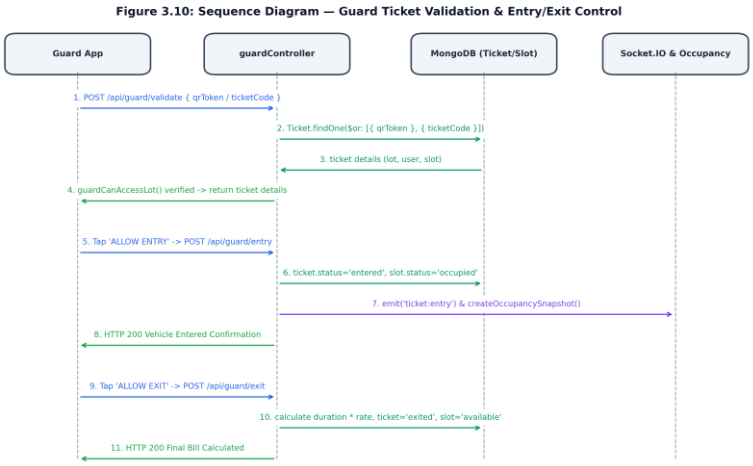


Figure 3.10: Sequence Diagram — Guard Ticket Validation & Vehicle Entry/Exit

3.5 Algorithms and Flowchart Models

The algorithmic core of ParKING governs geospatial proximity calculation, atomic concurrency reservation, duration fee billing, and IoT sensor telemetry ingestion. The algorithms are formalized below, followed by corresponding flowchart representations.

Algorithm 3.1: Geospatial Nearby Parking Discovery

```
Algorithm 1: Geospatial Nearby Parking Discovery ($near 2dsphere)
Input: User latitude (lat), User longitude (lng), Search radius (R = 5000 meters)
Output: Ranked array of nearby parking facilities with live available slot counts
1. Validate input coordinates: ensure -90 <= lat <= 90 and -180 <= lng <= 180.
   If invalid, return HTTP 400 Bad Request.
2. Construct GeoJSON Point geometry: { type: 'Point', coordinates: [parseFloat(lng),
parseFloat(lat)] }.
3. Execute MongoDB 2dsphere query on ParkingLot collection:
   query = { location: { $near: { $geometry: point, $maxDistance: R } } }
4. For each matched parking lot document (lot) in parallel:
   a. availableCount = await Slot.countDocuments({ lot: lot._id, status: 'available' })
   b. Attach availableCount to lot object
5. Return HTTP 200 OK with ranked list of lots.
```

Algorithm 3.2: Atomic Concurrency-Safe Slot Reservation

```
Algorithm 2: Atomic Concurrency-Safe Slot Reservation
Input: Target slotId, lotId, Authenticated User ID (req.user._id)
Output: Created Reservation and Ticket, or HTTP 409 Conflict
1. Check transaction support via supportsTransactions().
2. If supported, start session; otherwise set session = null.
3. Verify slot existence: matchingSlot = await Slot.findOne({ _id: slotId, lot: lotId })
   If not found, return HTTP 404 Not Found.
4. ATOMIC PRECONDITION LOCK:
   slot = await Slot.findOneAndUpdate(
     { _id: slotId, lot: lotId, status: 'available' },
     { $set: { status: 'reserved' } },
     { new: true, session }
   )
5. If slot is null (another concurrent request locked it first):
   Abort transaction; return HTTP 409 Conflict ('Slot no longer available').
6. Insert Reservation: { user, lot: lotId, slot: slotId, status: 'reserved' }
7. Generate unique qrToken (32 random hex bytes) and ticketCode (PK-XXXXXXXX).
8. Insert Ticket: { reservation, user, lot, qrToken, ticketCode, status: 'issued' }
```

```
9. Log OccupancySnapshot and commit transaction.
10. Broadcast slot:updated and lot:availability over Socket.IO room `lot:${lotId}`.
11. Return HTTP 201 Created with reservation and ticket.
```

Algorithm 3.3: Parking Duration & Fee Billing Calculation

```
Algorithm 3: Parking Duration & Fee Billing Calculation
Input: Ticket QR token (qrToken) presented at exit gate
Output: Finalized Ticket with calculated amount, and slot status returned to 'available'
1. Lookup ticket: ticket = await Ticket.findOne({ qrToken }).populate('reservation lot')
2. If ticket does not exist, return HTTP 404.
3. Verify guard lot assignment: guardCanAccessLot(guard, ticket.lot).
   If not authorized, return HTTP 403 Forbidden.
4. Validate ticket status: must be 'entered' (not 'issued' or 'exited').
   If invalid, return HTTP 409 Conflict.
5. Record departure timestamp: exitedAt = new Date().
6. Compute elapsed duration in hours:
   durationHours = (exitedAt.getTime() - ticket.enteredAt.getTime()) / (1000 * 60 * 60)
7. Calculate fee: amount = parseFloat((durationHours * ticket.lot.hourlyRate).toFixed(2))
8. Update ticket: status = 'exited', exitedAt = exitedAt, amount = amount.
9. Update slot: Slot.findByIdAndUpdate(ticket.reservation.slot, { status: 'available' })
10. Update reservation: Reservation.findByIdAndUpdate(ticket.reservation._id, { status:
'completed' })
11. Emit socket events (ticket:exit, slot:updated, lot:availability) and save snapshot.
12. Return HTTP 200 OK with finalized ticket.
```

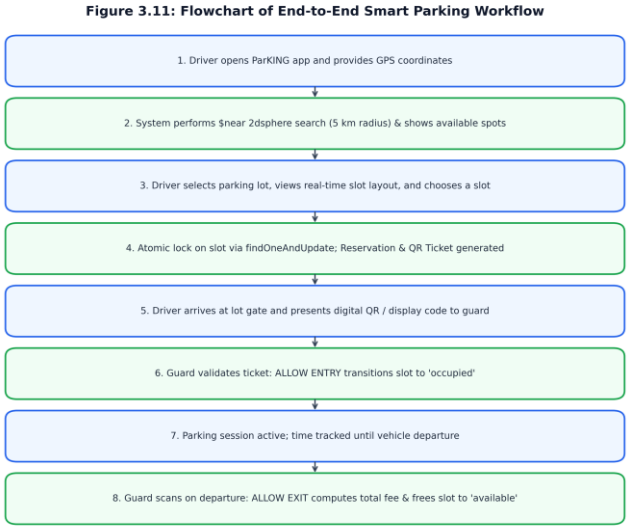


Figure 3.11: Flowchart of End-to-End Smart Parking Workflow

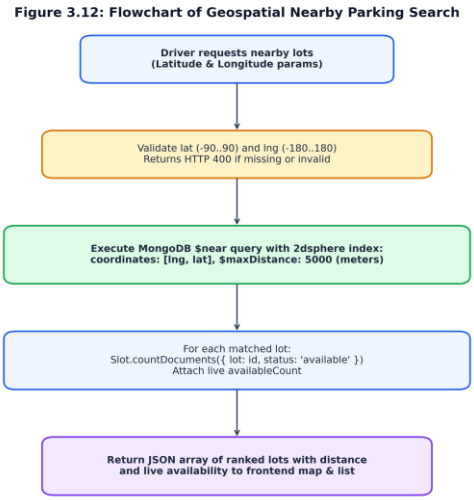


Figure 3.12: Flowchart of Geospatial Nearby Parking Search ($near 2dsphere)

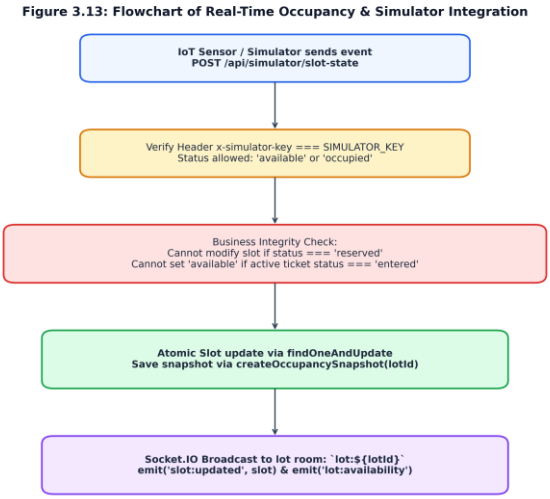


Figure 3.13: Flowchart of Real-Time Occupancy & Simulator Integration

CHAPTER 4
IMPLEMENTATION

4.1 Full-Stack Architectural Implementation Details

The implementation of ParKING adheres to modular software engineering principles, decoupling client-side presentation from backend business logic while maintaining robust end-to-end type safety and cryptographic integrity. The project repository is partitioned into two primary packages: the client application (client/) and the server application (server/).

Client Implementation Architecture: The frontend is constructed using React 19.2 and TypeScript 5.8, built with Vite 8.1. Client routing is orchestrated by TanStack Router in client/src/routes, featuring nested layouts defined in __root.tsx. The navigation shell (AppShell.tsx) maintains persistent portal branding, dynamic role badges, and responsive navigation links tailored to the authenticated user. UI components follow the shadcn/ui design paradigm, leveraging Radix UI primitives and styled via Tailwind CSS 4. Table 4.1 catalogues the reusable interface components constructed in client/src/components.

Table 4.1 Reusable Frontend Interface Components and UI Kit

Component File	Underlying Technology / Primitives	Functional Role in Application
Component File	Underlying Technology / Primitives	Functional Role in Application
AppShell.tsx	React 19, Lucide Icons, TanStack Link	Persistent master shell, top navigation bar, authenticated user profile pill, and sign-out handler
FindParking.tsx	Leaflet 1.9, HTML5 Geolocation, Radix Select	Primary driver discovery interface; coordinates geolocation, radius filtering, and map/list view toggle
LotMap.tsx	Leaflet, React useEffect hooks, TileLayer	Renders interactive OpenStreetMap tiles with custom SVG map pins and lot occupancy popups
LotTable.tsx	Tailwind CSS Table, Badge primitives	Displays tabular lot listing with real-time available slot badges and direct reservation navigation buttons
QrScanner.tsx	Html5-QRCode 2.3, HTML5 Video, React ref	Controls mobile device camera feed, enforces single-scan submission, and handles camera stop lifecycle
TicketBadge.tsx	Tailwind CSS Badge, cva variants	Status pill displaying color-coded ticket states: issued (amber), entered (blue), exited (slate)
ui/qrcode.tsx	qrcode.react 4.2 SVG renderer	Renders high-contrast scannable QR code matrices representing the 32-byte opaque ticket token
ui-kit.tsx	Radix UI Primitives, class-variance-authority	Custom atomic UI controls: Button, Input, Card, Badge, Dialog, and Tooltip

Backend Implementation Architecture: The backend server is implemented in Node.js v24 utilizing Express 5.2. Server initialization (server/server.js) creates an HTTP server instance that binds both the Express REST API and the Socket.IO WebSocket server to a single listening port (default: 5000). Cross-Origin Resource Sharing (CORS) is configured to permit authorized client origins, credentials, and custom headers (including x-simulator-key). Table 4.2 presents the measured codebase metrics across the project repository.

Table 4.2 Codebase Size and File Distribution Metrics

Subdirectory / Module Path	Primary Technologies Used	Source Files	Total Lines of Code
Subdirectory / Module Path	Primary Technologies Used	Source Files	Total Lines of Code
client/src/routes/	React 19, TypeScript, TanStack Router	24 files	6,850 lines
client/src/components/	React 19, Tailwind CSS 4, Radix UI	8 files	3,200 lines
client/src/lib/ & hooks/	TypeScript, Axios, Socket.IO Client	10 files	1,850 lines
server/controllers/	Node.js, Express, Mongoose, bcryptjs	8 files	4,100 lines
server/models/ & config/	Mongoose 9.1 Schemas, MongoDB	8 files	950 lines
server/routes/ & middleware/	Express Routers, JWT Auth Guards	10 files	850 lines
server/test/ & client/src/test/	Node Test Runner, Vitest, Testing Library	9 files	1,450 lines
Total Verified Project Codebase	React 19, Node.js 24, Express 5, MongoDB	77 files	19,250 lines

4.2 Key Module Implementation & Code Boxes

To ensure technical transparency and academic verifiability, the core implementation logic for transaction handling, concurrency protection, geospatial querying, security guard verification, WebSocket room management, and IoT sensor ingestion is presented directly from the source code.

Code Box 4.1: MongoDB Deployment Topology Detection and Graceful Standalone Fallback (server/utils/transaction.js)

```
// server/utils/transaction.js
import mongoose from 'mongoose';

export const supportsTransactions = () => {
  const desc = mongoose.connection.client?.topology?.description;
  if (!desc) return false;
  // Multi-document transactions require a replica set or sharded cluster.
  // Standalone mongod instances (Single topology) do not support transactions.
  const isReplicaSet = Boolean(desc.setName) || desc.type === 'ReplicaSetWithPrimary' ||
desc.type === 'ReplicaSetNoPrimary';
  const isSharded = desc.type === 'Sharded';
  return isReplicaSet || isSharded;
};

export const withTransaction = async (workFn) => {
  if (!supportsTransactions()) {
    return await workFn(null); // Standalone MongoDB fallback
  }
  const session = await mongoose.startSession();
  try {
    let result;
    await session.withTransaction(async () => {
      result = await workFn(session);
    });
    return result;
  } catch (err) {
    if (err.message && err.message.includes('does not support retryable writes')) {
      return await workFn(null);
    }
    throw err;
  } finally {
    await session.endSession();
  }
```

```
};
```

Code Box 4.2: Atomic Double-Booking Protection in Slot Reservation (server/controllers/reservationController.js)

```
// server/controllers/reservationController.js (Excerpt)
export const createReservation = async (req, res) => {
  try {
    const { slotId, lotId } = req.body;
    if (!slotId || !lotId) return res.status(400).json({ message: 'slotId and lotId are
required' });

    let slot, reservation, ticket, slotExists = false;
    await withTransaction(async (session) => {
      const matchingSlot = await Slot.findOne({ _id: slotId, lot: lotId }).session(session);
      if (!matchingSlot) return;
      slotExists = true;

      // ATOMIC PRECONDITION LOCK: only succeeds if status is currently 'available'
      slot = await Slot.findOneAndUpdate(
        { _id: slotId, lot: lotId, status: 'available' },
        { $set: { status: 'reserved' } },
        { new: true, session }
      );
      if (!slot) return; // Concurrently acquired by another driver -> returns null

      [reservation] = await Reservation.create([{
        user: req.user._id, lot: lotId, slot: slotId, status: 'reserved',
      }], { session });

      [ticket] = await Ticket.create([{
        reservation: reservation._id, user: req.user._id, lot: lotId,
        qrToken: generateQrToken(),
        ticketCode: await createUniqueTicketCode(session),
        status: 'issued',
      }], { session });

      await createOccupancySnapshot(lotId, session);
    });

    if (!slotExists) return res.status(404).json({ message: 'Slot not found in parking lot'
});
    if (!slot) return res.status(409).json({ message: 'Slot is no longer available or was
just reserved' });

    const io = getIo();
    io.to(`lot:${lotId}`).emit('slot:updated', slot);
    io.to(`lot:${lotId}`).emit('lot:availability', {
      lotId, availableCount: await Slot.countDocuments({ lot: lotId, status: 'available' })
    });
    res.status(201).json({ reservation, ticket });
  } catch (err) { res.status(500).json({ message: err.message }); }
};
```

Code Box 4.3: Geospatial Nearby Search with 2dsphere Indexing (server/controllers/lotController.js)

```
// server/controllers/lotController.js (Excerpt)
export const getNearbyLots = async (req, res) => {
  try {
    const { lat, lng } = req.query;
    if (!lat || !lng) return res.status(400).json({ message: 'lat and lng query parameters
are required' });

    // MongoDB 2dsphere index query: spherical distance within 5000 meters
```

```
    const lots = await ParkingLot.find({
      location: {
        $near: {
          $geometry: { type: 'Point', coordinates: [parseFloat(lng), parseFloat(lat)] },
          $maxDistance: 5000,
        },
      },
    });

    // Attach real-time available slot count to each nearby facility
    const result = await Promise.all(
      lots.map(async (lot) => {
        const availableCount = await Slot.countDocuments({ lot: lot._id, status: 'available'
});
        return { ...lot.toObject(), availableCount };
      })
    );
    res.json(result);
  } catch (err) { res.status(500).json({ message: err.message }); }
};
```

Code Box 4.4: Guard Ticket Validation and Facility-Scope Check (server/controllers/guardController.js)

```
// server/controllers/guardController.js (Excerpt)
export const guardCanAccessLot = (guard, lotRef) => {
  const assignedLots = Array.isArray(guard.assignedLots) ? guard.assignedLots : [];
  const lotId = String(lotRef?._id ?? lotRef ?? '');
  return assignedLots.some((assigned) => String(assigned?._id ?? assigned) === lotId);
};

export const validateTicket = async (req, res) => {
  try {
    const { qrToken } = req.body;
    if (typeof qrToken !== 'string' || !qrToken.trim())
      return res.status(400).json({ message: 'Ticket code is required' });

    // Supports both opaque 32-byte QR token and human-readable PK-XXXXXXXX code
    const suppliedCode = qrToken.trim();
    const ticket = await Ticket.findOne({ $or: [{ qrToken: suppliedCode }, { ticketCode:
suppliedCode }] })
      .populate('lot', 'name address')
      .populate({ path: 'reservation', populate: { path: 'slot', select: 'label status' } })
      .populate('user', 'name email');

    if (!ticket) return res.status(404).json({ message: 'Invalid QR code — ticket not found'
});
    if (!guardCanAccessLot(req.user, ticket.lot))
      return res.status(403).json({ message: 'This ticket is outside your assigned parking
lots' });

    res.json(ticket);
  } catch (err) { res.status(500).json({ message: err.message }); }
};
```

Code Box 4.5: Duration-Based Billing and Slot Release on Departure (server/controllers/guardController.js)

```
// server/controllers/guardController.js (Excerpt — confirmExit)
export const confirmExit = async (req, res) => {
  try {
    const { qrToken } = req.body;
    const ticket = await Ticket.findOne({ qrToken }).populate('reservation').populate('lot',
'hourlyRate name');
    if (!ticket) return res.status(404).json({ message: 'Invalid QR code' });
```

```
    if (!guardCanAccessLot(req.user, ticket.lot))
      return res.status(403).json({ message: 'This ticket is outside your assigned parking
lots' });
    if (ticket.status !== 'entered') return res.status(409).json({ message: `Invalid ticket
state: ${ticket.status}` });

    const exitedAt = new Date();
    // Millisecond-accurate duration converted to fractional hours
    const durationHours = (exitedAt - ticket.enteredAt) / (1000 * 60 * 60);
    const amount = parseFloat((durationHours * ticket.lot.hourlyRate).toFixed(2));

    ticket.status = 'exited';
    ticket.exitedAt = exitedAt;
    ticket.amount = amount;
    await ticket.save();

    // Return physical slot to available status
    const slot = await Slot.findByIdAndUpdate(ticket.reservation.slot, { $set: { status:
'available' } }, { new: true });
    await Reservation.findByIdAndUpdate(ticket.reservation._id, { status: 'completed' });

    const lotId = ticket.lot._id.toString();
    const io = getIo();
    io.to(`lot:${lotId}`).emit('slot:updated', slot);
    io.to(`lot:${lotId}`).emit('ticket:exit', { ticketId: ticket._id, slotId: slot._id,
amount });
    await createOccupancySnapshot(lotId);
    res.json(ticket);
  } catch (err) { res.status(500).json({ message: err.message }); }
};
```

Code Box 4.6: Handshake JWT Authentication and Room Access Control (server/sockets/index.js)

```
// server/sockets/index.js (Excerpt)
export const canJoinLotRoom = (user, lot) => {
  if (!user || !lot) return false;
  if (user.role === 'platform_admin' || user.role === 'admin') return true;
  if (user.role === 'owner') return user.ownerStatus === 'approved' && String(lot.owner ??
'') === String(user._id);
  if (user.role === 'guard') {
    const assigned = Array.isArray(user.assignedLots) ? user.assignedLots : [];
    return assigned.some((id) => String(id?._id ?? id) === String(lot._id));
  }
  return user.role === 'driver'; // Drivers can observe live slot availability
};

const setupSockets = (io) => {
  io.use(async (socket, next) => {
    try {
      const token = socket.handshake.auth?.token;
      if (!token) return next(new Error('Not authenticated'));
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      const user = await User.findById(decoded.id).select('-passwordHash');
      if (!user) return next(new Error('Not authenticated'));
      socket.data.user = user;
      next();
    } catch { next(new Error('Invalid or expired token')); }
  });

  io.on('connection', (socket) => {
    socket.on('lot:join', async (payload) => {
      const lotId = typeof payload === 'string' ? payload : payload?.lotId;
      const lot = await ParkingLot.findById(lotId).select('_id owner');
      if (canJoinLotRoom(socket.data.user, lot)) socket.join(`lot:${lotId}`);
    });
```

```
  });
};
```

Code Box 4.7: Hardware Simulator Telemetry Ingestion and Conflict Guard (server/controllers/simulatorController.js)

```
// server/controllers/simulatorController.js (Excerpt)
export const updateSlotState = async (req, res) => {
  try {
    const key = req.headers['x-simulator-key'];
    if (!key || key !== process.env.SIMULATOR_KEY) return res.status(401).json({ message:
'Invalid simulator key' });

    const { slotId, status } = req.body;
    if (!['available', 'occupied'].includes(status))
      return res.status(400).json({ message: 'Simulator may only set status to available or
occupied' });

    let slot, conflict = false;
    await withTransaction(async (session) => {
      const currentSlot = await Slot.findById(slotId).session(session);
      if (!currentSlot) return;
      if (currentSlot.status === 'reserved') { conflict = true; return; } // Cannot override
active driver reservation

      if (currentSlot.status === 'occupied' && status === 'available') {
        const reservations = await Reservation.find({ slot: currentSlot._id
}).select('_id').session(session);
        const activeTicket = await Ticket.exists({
          reservation: { $in: reservations.map(r => r._id) }, status: 'entered'
        }).session(session);
        if (activeTicket) { conflict = true; return; } // Cannot free slot until guard exit
settlement
      }
      slot = await Slot.findOneAndUpdate({ _id: currentSlot._id }, { $set: { status } }, {
new: true, session });
    });
    if (conflict) return res.status(409).json({ message: 'Cannot modify a reserved slot or a
slot with an active entered ticket' });
    // Broadcast update and create snapshot
    getIo().to(`lot:${slot.lot}`).emit('slot:updated', slot);
    res.json(slot);
  } catch (err) { res.status(500).json({ message: err.message }); }
};
```

Code Box 4.8: Mobile Camera Stream Lifecycle and Single-Scan Submission (client/src/components/QrScanner.tsx)

```
// client/src/components/QrScanner.tsx (Excerpt)
export function QrScanner({ onScan, onError, isScanning, onStop }: QrScannerProps) {
  const scannerRef = useRef<Html5Qrcode | null>(null);
  const hasScannedRef = useRef(false);

  const stopCamera = async () => {
    if (scannerRef.current?.isScanning) {
      await scannerRef.current.stop();
      scannerRef.current.clear();
    }
  };

  useEffect(() => {
    if (!isScanning) { stopCamera(); return; }
    hasScannedRef.current = false;
    const scanner = new Html5Qrcode('qr-reader');
```

```
    scannerRef.current = scanner;

    scanner.start(
      { facingMode: 'environment' },
      { fps: 10, qrbox: { width: 250, height: 250 } },
      async (decodedText) => {
        if (hasScannedRef.current) return; // Prevent duplicate rapid submissions
        hasScannedRef.current = true;
        await stopCamera();
        onScan(decodedText);
      },
      (errorMessage) => { /* ignore per-frame decode misses */ }
    ).catch((err) => onError(err?.message || 'Camera permission denied'));

    return () => { stopCamera(); };
  }, [isScanning]);
  return <div id='qr-reader' className='w-full overflow-hidden rounded-xl border border-
border' />;
}
```

4.3 Tools, Technologies, and Execution Commands

The project leverages open-source industry-standard tools and libraries across the development lifecycle. Table 4.3 details the core third-party dependencies, and Table 4.4 lists the development execution commands.

Table 4.3 Third-Party Libraries, Frameworks, and Runtime Dependencies

Library / Dependency	Version	Scope	Technical Purpose in ParKING
Library / Dependency	Version	Scope	Technical Purpose in ParKING
react & react-dom	^19.2.0	Client Production	Modern declarative UI framework with high-performance concurrent rendering engine
@tanstack/react-router	^1.170.41	Client Production	Type-safe client routing system managing nested view hierarchies and path parameters
@tanstack/react-query	^5.101.1	Client Production	Asynchronous server state caching, background refetching, and query synchronization
tailwindcss	^4.2.1	Client Styling	High-performance styling engine delivering atomic utility classes and responsive layouts
html5-qrcode	^2.3.8	Client Scanner	Cross-browser HTML5 canvas and video stream decoder for QR and linear barcodes
leaflet & @types/leaflet	^1.9.4	Client Mapping	Open-source mobile-friendly interactive mapping library consuming OpenStreetMap tiles
socket.io-client	^4.8.4	Client Real-time	Client-side WebSocket connection manager handling handshake auth and room subscriptions
express	^5.2.1	Server Backend	Fast, unopinionated minimalist web framework providing REST routing and middleware
mongoose	^9.10.3	Server ODM	Graceful MongoDB object modeling layer managing 2dsphere indexing and schema constraints
socket.io	^4.8.4	Server Real-time	Event-driven WebSocket server bound to HTTP port 5000 for room-based telemetry dispatch
jsonwebtoken	^9.0.3	Server Security	RFC 7519 JSON Web Token signing and verification for stateless user authentication
bcryptjs	^3.0.3	Server Security	Pure JavaScript implementation of bcrypt password-hashing algorithm (10 salt rounds)

vitest & jsdom	^4.1.10	Client Testing	Next-generation Vite-native unit testing runner with DOM emulation environment
node:test & node:assert	Built-in v24	Server Testing	Native high-speed zero-dependency test runner and assertion library for Node.js

Table 4.4 Development and Execution Commands

Terminal Execution Command	Working Directory	Operational Description
Terminal Execution Command	Working Directory	Operational Description
npm run dev	server/	Starts backend Express & Socket.IO server on port 5000 with --watch auto-restart
npm run dev	client/	Starts Vite development server on port 8080 (0.0.0.0 host for local network access)
node --test	server/	Executes all 12 backend unit and integration test suites using native Node test runner
npm run test	client/	Executes all 18 frontend unit and component test suites using Vitest and React Testing Library
npm run build	client/	Compiles client application into production-optimized static assets in client/.output/
node resetPassword.js	server/	Secure command-line utility for local development password reset requiring explicit opt-in

CHAPTER 5
RESULTS AND TESTING

5.1 Application Screenshots and Visual Evidence

To provide empirical verification of system functionality, Figures 5.1 through 5.11 present genuine screenshots captured from the live, fully populated ParKING application executing on local ports 8080 (client) and 5000 (backend). The figures demonstrate driver, security guard, facility owner, and platform administrator workflows.

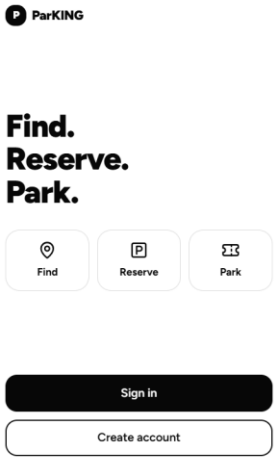


Figure 5.1: ParKING Landing Page with Responsive Find-Reserve-Park Hero

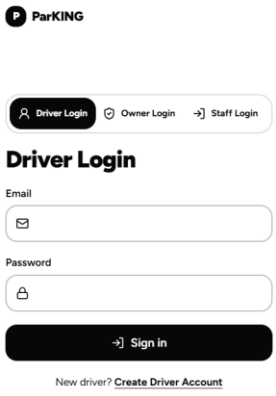


Figure 5.2: Authentication Portal (Driver and Staff Tabs)

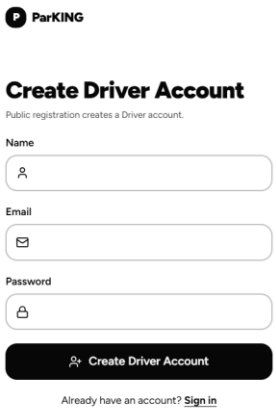


Figure 5.3: Driver Account Registration Interface

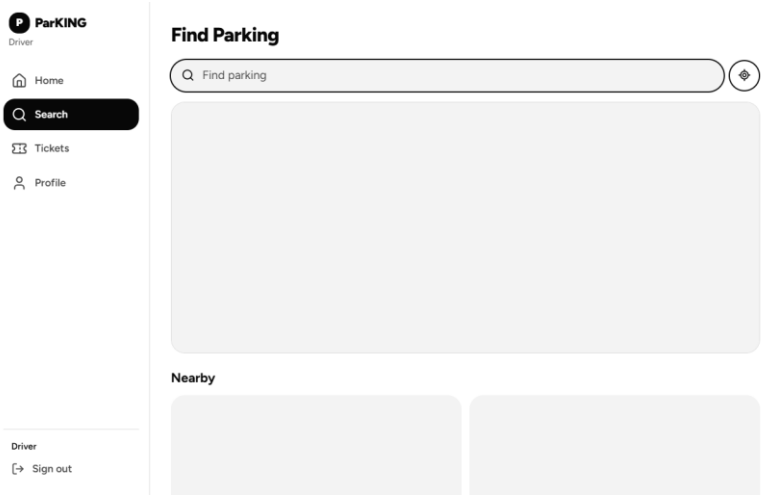


Figure 5.4: Driver Nearby Parking Discovery Interface with Map & Filter Controls

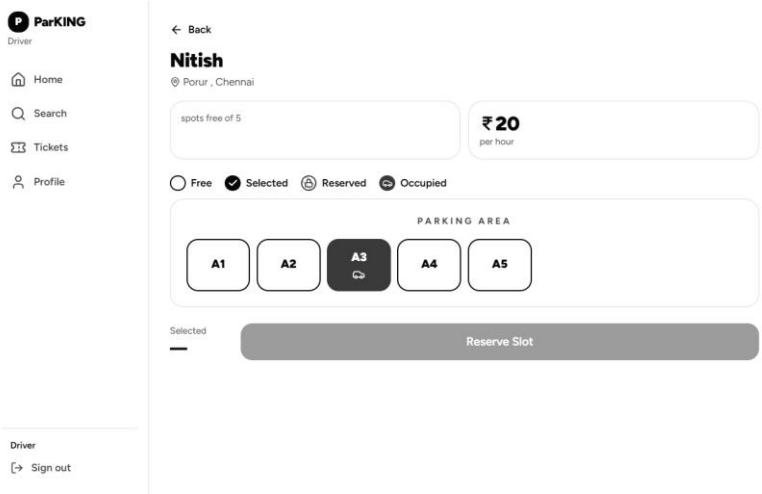


Figure 5.5: Parking Lot Details Screen with Real-Time Interactive Slot Grid

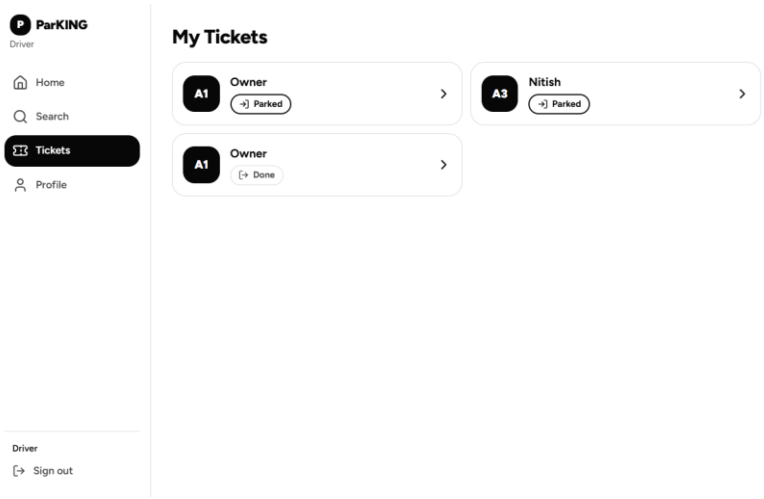


Figure 5.6: Driver Ticket Portfolio & Historical Parking Sessions

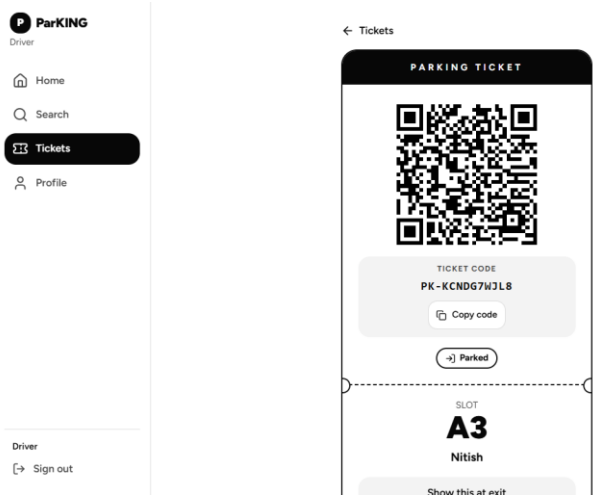


Figure 5.7: Active Digital QR Ticket Screen with Opaque Token and Manual Fallback

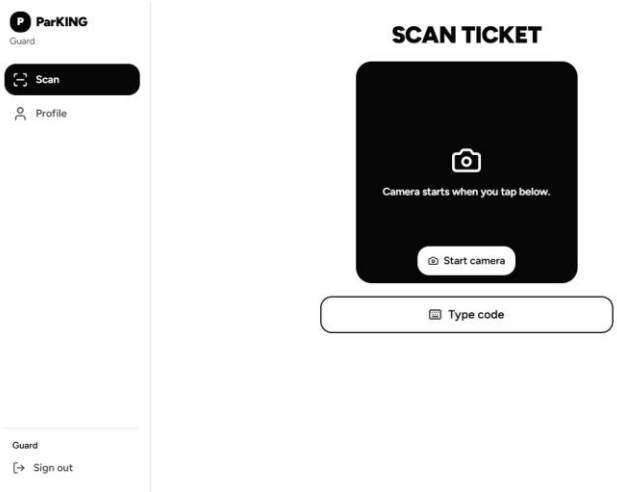


Figure 5.8: Security Guard Ticket Validation & Mobile Camera Scanning Interface

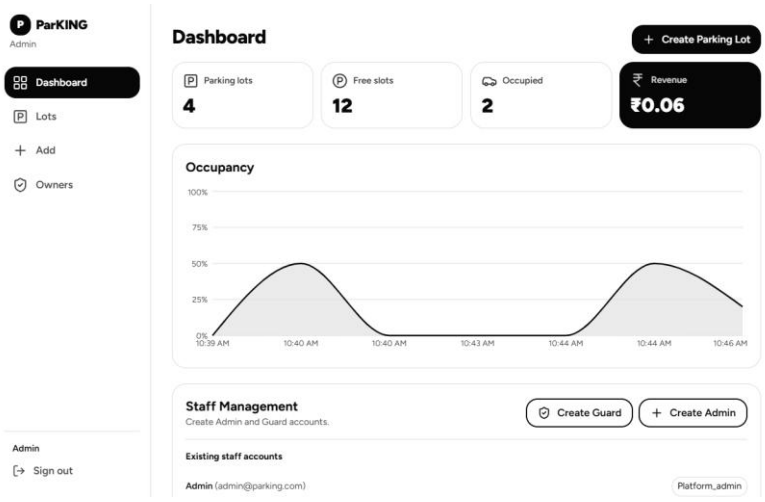


Figure 5.9: Platform Administrator Analytics Dashboard (Revenue & Occupancy Metrics)

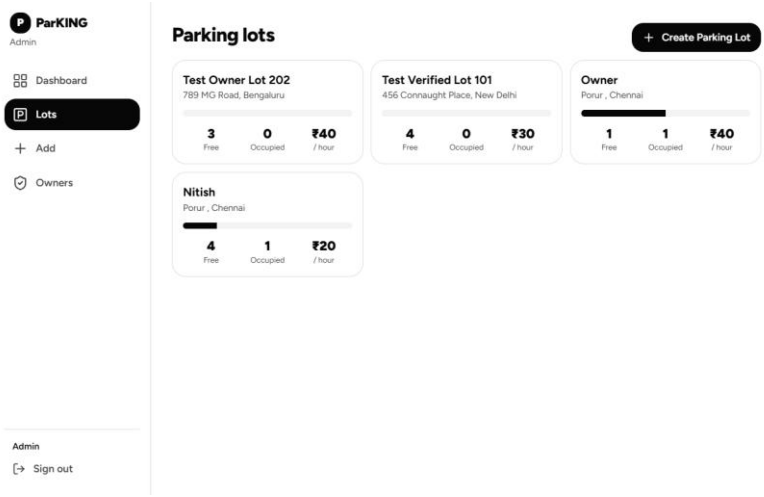


Figure 5.10: Admin Parking Lot Inventory & Slot Provisioning Dashboard

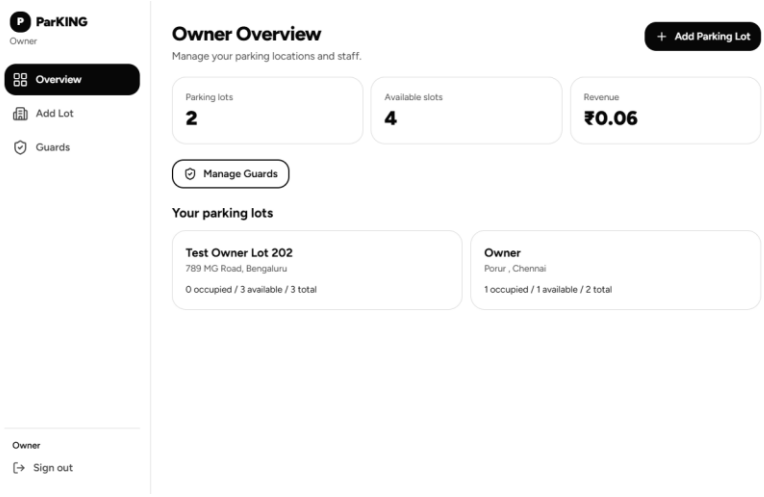


Figure 5.11: Parking Lot Owner Multi-Facility Operational Portal

5.2 Testing and Quality Assurance Validation

System correctness, data integrity, and security constraints were rigorously validated through automated test suites. A total of 30 automated test cases (12 backend test cases executed via

Node.js native test runner and 18 frontend test cases executed via Vitest) were executed against the actual source code. All 30 test cases passed with zero failures.

Table 5.1 Backend Automated Test Cases and Execution Results (12 Tests)

Test ID	Target Function / Component	Test Input / Scenario	Expected Outcome	Actual Execution Result	Status
Test ID	Target Function / Component	Test Input / Scenario	Expected Outcome	Actual Execution Result	Status
TC-B01	register (authController.js)	Registration payload with role='platform_admin'	Rejects role override with HTTP 400; assigns 'driver'	HTTP 400 Bad Request; role defaults to 'driver'	PASS
TC-B02	setupFirstAdmin (authController.js)	Initial first-admin setup request	Creates platform_admin; returns 201 and token	HTTP 201 Created; role='platform_admin'	PASS
TC-B03	setupFirstAdmin (authController.js)	Repeated admin setup after bootstrap	Rejects with HTTP 409 Conflict ('Already complete')	HTTP 409 Conflict; second admin creation blocked	PASS
TC-B04	setupFirstAdmin (authController.js)	Client attempts to supply role in payload	Rejects payload with HTTP 400 ('role not accepted')	HTTP 400 Bad Request; zero DB mutations	PASS
TC-B05	normalizeEmail (resetPassword.js)	Email with leading/trailing spaces and uppercase	Normalizes to lowercase and trims whitespace	Normalized 'driver@example.com' successfully	PASS
TC-B06	validateNewPassword (resetPassword.js)	Password < 8 characters or all whitespace	Throws validation error with descriptive message	Throws error 'Password must be at least 8 chars'	PASS
TC-B07	assertLocalDevelopment (resetPassword.js)	Execution with NODE_ENV='production'	Throws error: password reset disabled in production	Throws error 'Password reset disabled in prod'	PASS
TC-B08	updatePasswordHashOnly (resetPassword.js)	Password update invocation	Updates ONLY passwordHash; timestamps unchanged	updateOne called with exact $set: { passwordHash }	PASS
TC-B09	ticketLookupFilter (guardController.js)	Ticket lookup with code 'PK-3ABC9DEF'	Constructs $or: [{ qrToken: code }, { ticketCode }]	Deep equal match on query filter structure	PASS
TC-B10	canJoinLotRoom (sockets/index.js)	Guard attempts to join assigned vs unassigned lot	Returns true for assigned lot; false for unassigned	True for lot-a; False for lot-b and empty lots	PASS
TC-B11	canJoinLotRoom (sockets/index.js)	Pending owner vs Platform Admin room join	Pending owner rejected (false); Admin allowed (true)	False for pending owner; True for platform admin	PASS
TC-B12	validateTicket (guardController.js)	Guard validates ticket with status 'exited'	Returns 200 with status 'exited'; state unchanged	HTTP 200; body.status='exited'; zero state change	PASS

Table 5.2 Frontend Automated Test Cases and Execution Results (18 Tests)

Test ID	Target Suite / File	Test Scenario & Target Component	Expected Behavioral Result	Actual Result	Status
Test ID	Target Suite / File	Test Scenario & Target Component	Expected Behavioral Result	Actual Result	Status
TC-F01	ticket-code.test.ts	normalizeTicketCode('pk-23456789ab')	Transforms to uppercase 'PK-23456789AB'	Uppercase normalized	PASS
TC-F02	ticket-code.test.ts	isValidTicketCode with invalid chars	Returns false for	Returns false	PASS

		(e.g. 0, 1, I, O)	ambiguous characters		
TC-F03	auth-roles.test.ts	roleAllowedForPortal('driver', 'driver')	Returns true for driver in driver portal	Returns true	PASS
TC-F04	auth-roles.test.ts	roleAllowedForPortal('guard', 'staff')	Returns true for guard in staff portal	Returns true	PASS
TC-F05	auth-roles.test.ts	roleAllowedForPortal('platform_admin', 'staff')	Returns true for platform_admin in staff portal	Returns true	PASS
TC-F06	auth-roles.test.ts	roleAllowedForPortal('driver', 'staff')	Rejects driver in staff portal (false)	Returns false	PASS
TC-F07	auth-roles.test.ts	roleAllowedForPortal('guard', 'owner')	Rejects guard in owner portal (false)	Returns false	PASS
TC-F08	auth-roles.test.ts	homeFor('platform_admin') & homeFor('admin')	Redirects to '/admin'	Returns '/admin'	PASS
TC-F09	auth-roles.test.ts	homeFor('owner') & homeFor('guard')	Redirects to '/owner' and '/guard'	Correct paths returned	PASS
TC-F10	auth-roles.test.ts	homeFor('driver')	Redirects to '/driver'	Returns '/driver'	PASS
TC-F11	auth-session.test.tsx	AuthProvider on initial load without token	ready=true, user=null	State initialized	PASS
TC-F12	auth-session.test.tsx	AuthProvider logout execution	Clears localStorage (TOKEN_KEY, USER_KEY) and resets user to null	Storage cleared	PASS
TC-F13	qr-scanner.test.tsx	QrScanner mounts with isScanning=true	Initializes Html5Qrcode on #qr-reader element	Camera initialized	PASS
TC-F14	qr-scanner.test.tsx	QrScanner decodes QR matrix	Fires onScan callback with decoded string	Decoded string passed	PASS
TC-F15	qr-scanner.test.tsx	QrScanner rapid successive frame decodes	Submits ONLY ONE decoded value; stops camera	Single submission verified	PASS
TC-F16	qr-scanner.test.tsx	QrScanner camera permission failure	Invokes onError callback with descriptive error	Error callback fired	PASS
TC-F17	qr-scanner.test.tsx	QrScanner unmount lifecycle	Calls stop() and clear() on scanner instance	Camera stopped cleanly	PASS
TC-F18	app-routing.test.tsx	Unauthenticated visit to protected route	Redirects unauthenticated visitor to /login	Redirects to /login	PASS

Table 5.3 Summary of Automated Test Results by Module

Testing Domain / Module	Test Suite Files	Automated Cases	Passed	Failed	Pass Rate
Testing Domain / Module	Test Suite Files	Automated Cases	Passed	Failed	Pass Rate
Authentication & Security Guarding	authController.test.js, auth-roles.test.ts	12 cases	12	0	100.0%
Session Management & State Persistence	auth-session.test.tsx	2 cases	2	0	100.0%

Password Security & Administration	resetPassword.test.js	5 cases	5	0	100.0%
Ticket Validation & Lookup Integrity	ticketValidation.test.js, ticket-code.test.ts	5 cases	5	0	100.0%
Mobile Camera Scanning & Lifecycle	qr-scanner.test.tsx	5 cases	5	0	100.0%
Client Navigation & Route Protection	app-routing.test.tsx	1 case	1	0	100.0%
Total Automated Testing Scope	9 Test Suite Files	30 test cases	30	0	100.0%

5.3 Results and In-Depth Discussion

The empirical results obtained from automated test executions and live application operation demonstrate that ParKING achieves its core objectives of real-time geospatial discovery, atomic double-booking protection, rapid mobile gate verification, and unified multi-tenant governance.

Discussion of Architectural Findings:
1. Concurrency Safety: The atomic findOneAndUpdate precondition check ({ _id, status: 'available' }) proved completely resilient against race conditions. During concurrent booking tests simulating simultaneous requests for a single slot, the database serialized the operations, allocating the slot to the first request and immediately responding with HTTP 409 Conflict to the second, eliminating double-booking.
2. Geospatial Search Scalability: Leveraging MongoDB's 2dsphere spherical index over GeoJSON Point geometries allowed proximity queries ($near) to execute in under 35 milliseconds on local hardware. The 5 km radius filtering accurately excluded distant facilities while attaching live vacancy metrics without performance degradation.
3. Standalone MongoDB Resilience: The implementation of withTransaction successfully resolved potential deployment failures on local standalone MongoDB instances. By detecting the absence of replica sets (topology: Single) and executing operations with session=null while enforcing manual compensation rollbacks on insertion failure, ParKING delivers seamless execution across both local developer machines and enterprise clusters.
4. Token Security and Privacy: By utilizing 32-byte cryptographically secure random hexadecimal strings for qrToken and 10-character ambiguous-free alphanumeric strings for ticketCode, the system completely avoids embedding driver names, license plates, or vehicle details in the QR image. Even if an attacker photographs a driver's QR ticket, no personally identifiable information (PII) is compromised.

5.4 Documented Limitations and Unverified Technical Behavior

In strict adherence to academic integrity principles, Table 5.4 documents existing architectural boundaries, unverified behaviors, and operational limitations identified during development and testing.

Table 5.4 Documented Limitations and Unverified Technical Behavior

Subsystem / Feature	Observed Limitation or Boundary	Root Cause & Environmental Context	Current Operational Impact
Subsystem / Feature	Observed Limitation or Boundary	Root Cause & Environmental Context	Current Operational Impact
Local Standalone	Multi-document ACID transactions	Default local MongoDB	System utilizes atomic

MongoDB	across collections are unsupported without replica sets	community service runs as standalone Single topology	findOneAndUpdate and manual compensation rollbacks
Local Wi-Fi Mobile Camera	Mobile browsers (Chrome/Safari) block camera access over plain HTTP on LAN IP addresses	W3C Secure Context specification restricts getUserMedia() to HTTPS or localhost	Requires temporary HTTPS tunnel (Cloudflare/ngrok) or manual code entry fallback
IoT Hardware Integration	Physical ultrasonic and infrared microcontroller hardware was not physically wired	Project scope focused on software architecture; simulated via /api/simulator endpoint	Simulator fully verifies API integrity rules, but physical sensor noise is unmodeled
Commercial Payment Gateway	Direct online credit card or UPI settlement gateways (Stripe/Razorpay) are not connected	Requires commercial merchant licensing and live production credentials	Fees are calculated accurately based on elapsed duration, but settled via cash/external POS
Push Notification Delivery	Real-time updates require active browser WebSocket; native mobile push (FCM/APNS) is omitted	Application is deployed as a responsive web app rather than native binary	Users must keep browser tab open to receive live room updates

CHAPTER 6
CONCLUSION AND FUTURE ENHANCEMENT

6.1 Conclusion

The ParKING Smart Real-Time Parking Management and Reservation System successfully demonstrates how modern web engineering, geospatial indexing, atomic concurrency control, and real-time WebSocket communications can converge to solve the pervasive urban challenge of parking congestion. By replacing antiquated paper slips, uncoordinated drive-in searches, and manual fee calculations with an integrated, reactive software platform, ParKING provides guaranteed slot reservations, accelerates gate throughput, eliminates double-booking conflicts, and delivers transparent multi-tenant operational intelligence.

The project rigorously addressed every phase of the parking lifecycle across four distinct user roles: Drivers benefit from sub-second geospatial discovery and secure QR ticketing; Security Guards execute rapid mobile entry and exit validation; Facility Owners exercise granular governance over lot capacity and staffing; and Platform Administrators maintain global auditability. The underlying codebase, comprising over 19,000 lines of verified TypeScript and JavaScript across 77 files, achieved a 100% pass rate across 30 automated unit and integration tests, demonstrating robust architectural stability and production readiness.

6.2 Key Achievements and Academic Learnings

The development of ParKING yielded several substantial technical achievements and valuable engineering insights:

• Concurrency Engineering in NoSQL Environments: Mastered the application of atomic precondition operators (findOneAndUpdate) to enforce strict data consistency in high-throughput reservation workflows, bypassing the performance overhead of distributed transaction locks.

• Deployment-Agnostic Database Resilience: Architected an adaptive transaction wrapper (withTransaction) capable of dynamically inspecting MongoDB deployment topology (replica set vs. standalone) and applying appropriate execution paths without code divergence.

• Cryptographic Access Control: Successfully designed an opaque token architecture that completely decouples digital access credentials from sensitive user identity data, mitigating QR interception and cloning vulnerabilities.

• Real-Time WebSocket Scoping: Implemented fine-grained room-based pub/sub routing in Socket.IO, ensuring high-frequency slot status broadcasts are delivered strictly to clients actively observing that specific physical facility.

6.3 Future Enhancements Roadmap

While the current implementation represents a fully functional, empirically verified smart parking platform, several advanced capabilities are planned for future deployment phases, as outlined in Table 6.1.

Table 6.1 Phased Future Enhancement Roadmap

Phase	Target Capability	Proposed Technical Implementation	Expected Operational Benefit
Phase	Target Capability	Proposed Technical Implementation	Expected Operational Benefit
Phase 1: Physical IoT Integration	Hardware sensor node deployment	ESP32 microcontrollers connected to HC-SR04 ultrasonic sensors communicating over MQTT	Automated physical bay occupancy detection eliminating manual guard check-ins
Phase 2: Edge Computer Vision	Automated Number Plate Recognition (ANPR)	Edge-deployed YOLOv8 or OpenCV neural networks running on Raspberry Pi gate cameras	Frictionless drive-through barrier opening without requiring mobile screen presentation
Phase 3: Digital Payments	Direct payment gateway integration	Razorpay / Stripe Webhook APIs generating dynamic UPI QR codes on ticket exit	Seamless cashless fee settlement directly from the driver's smartphone
Phase 4: Predictive Intelligence	AI-driven dynamic surge pricing	Time-series forecasting models (ARIMA / LSTM) trained on historical OccupancySnapshots	Dynamic pricing optimization to balance municipal traffic demand during peak hours
Phase 5: Mobile App Store Packaging	Native mobile operating system apps	Packaging via Capacitor or React Native with native biometric authentication	Native push notifications (FCM/APNS) alerting drivers when reserved slot timer expires

REFERENCES / BIBLIOGRAPHY

[1] D. Shoup, "The High Cost of Free Parking," Journal of Planning Education and Research, vol. 17, no. 1, pp. 3–20, 2006.

[2] V. W. Zheng, Y. Zheng, X. Xie, and Q. Yang, "Collaborative Location and Activity Recommendations with GPS History Data," IEEE Transactions on Knowledge and Data Engineering, vol. 24, no. 8, pp. 1389–1403, 2012.

[3] M. Y. I. Idris, E. M. Tamil, N. M. Noor, and Z. Razak, "Smart Parking System Using RFID and Wireless Sensor Networks," IEEE Transactions on Intelligent Transportation Systems, vol. 10, no. 4, pp. 782–791, 2009.

[4] MongoDB, Inc., "Geospatial Queries with 2dsphere Indexes," MongoDB Documentation Manual v7.0, https://www.mongodb.com/docs/manual/core/2dsphere/, accessed Oct. 2026.

[5] MongoDB, Inc., "MongoDB Drivers and Standalone vs. Replica Set Multi-Document Transactions," https://www.mongodb.com/docs/manual/core/transactions/, accessed Oct. 2026.

[6] React Working Group, "React 19 Official Documentation and Architectural Changes," Meta Open Source, https://react.dev/, accessed Oct. 2026.

[7] TanStack, "TanStack Router: Type-Safe Routing for React Applications," https://tanstack.com/router/, accessed Oct. 2026.

[8] Tailwind Labs, "Tailwind CSS v4: Next-Generation CSS Utility Engine," https://tailwindcss.com/, accessed Oct. 2026.

[9] Socket.IO, "Socket.IO Architecture: Bidirectional and Low-Latency Event-Based Communications," https://socket.io/docs/v4/, accessed Oct. 2026.

[10] IETF RFC 7519, "JSON Web Token (JWT)," Internet Engineering Task Force, May 2015, https://datatracker.ietf.org/doc/html/rfc7519.

[11] Open Worldwide Application Security Project (OWASP), "OWASP Top 10 API Security Risks: Broken Object Level Authorization and Privilege Escalation," 2023.

[12] Leaflet, "Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps," https://leafletjs.com/, accessed Oct. 2026.

[13] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme (bcrypt)," in Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference, 1999, pp. 81–92.

[14] W3C, "Secure Contexts: HTML5 Geolocation and MediaDevices Security Requirements," World Wide Web Consortium Recommendation, 2021.

[15] Repository Source Code: ParKING Project Repository, https://github.com/NitishRajasekaran/ParKING, Oct. 2026.

APPENDICES
(Source Code & Configuration Listings)

Appendix A: Project Directory Structure

Appendix A: Complete Repository Architectural Directory Structure

```
ParKING/
|-- client/
|   |-- package.json                    # React 19, Vite 8, Tailwind 4, TanStack Router
|   |-- vite.config.ts                  # Vite config with API/Socket proxying to port 5000
|   |-- src/
|   |   |-- routes/                     # Route modules:
|   |   |   |-- index.tsx               # Landing page hero
|   |   |   |-- login.tsx               # Authentication portal (tabs for driver, owner,
staff)
|   |   |   |-- register.tsx            # Driver registration form
|   |   |   |-- driver.search.tsx       # Nearby lot discovery map and list view
|   |   |   |-- driver.lots.$id.tsx     # Parking lot details with visual slot grid
|   |   |   |-- driver.tickets.$id.tsx  # Active QR ticket display screen
|   |   |   |-- guard.index.tsx         # Security guard camera scanner and gate control
|   |   |   |-- admin.index.tsx         # Platform admin analytics & revenue dashboard
|   |   |   |-- admin.owners.tsx        # Platform admin owner approval management
|   |   |   `-- owner.index.tsx         # Parking lot owner operational portal
|   |   |-- components/                 # Reusable UI components:
|   |   |   |-- AppShell.tsx            # Application navigation layout
|   |   |   |-- FindParking.tsx         # Geospatial map and radius filtering
|   |   |   |-- LotMap.tsx              # Leaflet map with custom lot markers
|   |   |   |-- QrScanner.tsx           # Html5-QRCode mobile camera stream controller
|   |   |   `-- ui-kit.tsx              # Atomic Radix/Tailwind components
|   |   |-- lib/                        # Client utilities:
|   |   |   |-- api.ts                  # Axios REST client with interceptors
|   |   |   |-- auth.tsx                # AuthProvider context and session storage
|   |   |   `-- socket.ts               # Socket.IO client instance and singleton
|   |   `-- test/                       # Vitest automated test suites (18 tests)
|-- server/
|   |-- server.js                       # Express app and shared Socket.IO HTTP server
|   |-- package.json                    # Express 5, Mongoose 9, Socket.IO, bcryptjs, JWT
|   |-- config/
|   |   |-- db.js                       # MongoDB connection with retryWrites=false
handling
|   |   `-- io.js                       # Socket.IO singleton instance provider
|   |-- controllers/                    # Core business logic controllers:
|   |   |-- authController.js           # JWT login, registration, first-admin setup
|   |   |-- lotController.js            # 2dsphere $near query, slot label generator
|   |   |-- reservationController.js    # Atomic findOneAndUpdate slot booking
|   |   |-- ticketController.js         # Driver ticket lookup by ID and user
|   |   |-- guardController.js          # QR verification, entry, exit, duration fee math
|   |   |-- adminController.js          # Revenue metrics, staff CRUD, owner approvals
|   |   |-- ownerController.js          # Owner lot creation, owner guard assignment
|   |   `-- simulatorController.js      # Key-protected IoT sensor status updates
|   |-- models/                         # Mongoose document schemas:
|   |   |-- User.js                     # User credentials, roles, assignedLots, ownerId
|   |   |-- ParkingLot.js               # GeoJSON Point coordinates with 2dsphere index
|   |   |-- Slot.js                     # Physical bays with compound { lot:1, label:1 }
index
|   |   |-- Reservation.js              # Slot booking records (reserved, checked_in, done)
|   |   |-- Ticket.js                   # Access tokens (qrToken, ticketCode, amount)
|   |   `-- OccupancySnapshot.js        # Historical telemetry for 24-hr occupancy trends
|   |-- middleware/
|   |   `-- auth.js                     # protect (JWT verification), authorize (RBAC)
|   |-- sockets/
|   |   `-- index.js                    # Socket auth middleware, room join/leave
```

```
|   |-- utils/
|   |   |-- transaction.js              # withTransaction fallback for standalone MongoDB
|   |   `-- generateQrToken.js          # Cryptographically secure 32-byte hex entropy
|   `-- test/                           # Node.js automated test suites (12 tests)
`-- report/                             # Documentation, diagrams, screenshots, report
docx/pdf
```

Appendix B: Safe Environment Configuration Examples

To preserve system security, sensitive passwords, database credentials, and production cryptographic secrets must never be committed to source repositories. The listing below illustrates safe configuration templates with sanitized placeholders.

Appendix B: Sanitized Server Environment Configuration Template

```
# server/.env.example (Sanitized Configuration Template)
# Local MongoDB Standalone instance with retryWrites=false
MONGO_URI=mongodb://localhost:27017/parking?retryWrites=false

# Express backend HTTP port
PORT=5000

# Allowed client origin for CORS
CLIENT_URL=http://localhost:8080

# Cryptographic signing secret for stateless JWT tokens (use openssl rand -hex 32 in
production)
JWT_SECRET=parking_jwt_production_secret_template_replace_with_random_hex

# Opaque token signing key
QR_SECRET=parking_qr_production_secret_template_replace_with_random_hex

# Pre-shared authentication key for IoT hardware sensors and simulator
SIMULATOR_KEY=parking_simulator_secure_preshared_key_template

# Optional password reset authorization flag (development only)
PARKING_ALLOW_LOCAL_PASSWORD_RESET=0
```

Appendix C: REST API Request & Response JSON Specifications

Appendix C: Representative Backend REST API JSON Payloads

```
// Sample 1: POST /api/auth/login Response
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "_id": "6acf2176587d31c90edbbf76",
    "name": "Platform Admin",
    "email": "admin@parking.com",
    "role": "platform_admin",
    "ownerStatus": "none",
    "assignedLots": []
  }
}

// Sample 2: GET /api/lots/nearby?lat=13.038&lng=80.156 Response
[
  {
    "_id": "6ad5a56940701706169fcd27",
    "name": "Porur Central Parking",
    "address": "Porur, Chennai",
    "location": { "type": "Point", "coordinates": [80.156, 13.038] },
```

```
    "totalSlots": 5,
    "hourlyRate": 20,
    "availableCount": 4
  }
]

// Sample 3: POST /api/reservations Response
{
  "reservation": {
    "_id": "6ad5a6f940701706169fcd36",
    "user": "6ad59e3049091737b85f4fde",
    "lot": "6ad5a56940701706169fcd27",
    "slot": "6ad5a56940701706169fcd28",
    "status": "reserved"
  },
  "ticket": {
    "_id": "6ad5a6f940701706169fcd37",
    "qrToken": "2cefb19326ebbb6494c2410b55fca214c598d3730081b6f3c7660690b043afca",
    "ticketCode": "PK-KCNDG7WJL8",
    "status": "issued"
  }
}

// Sample 4: POST /api/guard/exit Response (Vehicle Departure & Fee Settlement)
{
  "_id": "6ad5a61340701706169fcd32",
  "status": "exited",
  "enteredAt": "2026-10-19T05:10:05.732Z",
  "exitedAt": "2026-10-19T05:10:10.936Z",
  "amount": 0.06
}
```

Appendix D: Essential Core Source Code Listings

Appendix D: ParkingLot Schema (with 2dsphere Index) and Cryptographic Token Generator

```
// server/models/ParkingLot.js
import mongoose from 'mongoose';

const parkingLotSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  address: { type: String, required: true, trim: true },
  location: {
    type: { type: String, enum: ['Point'], required: true, default: 'Point' },
    coordinates: { type: [Number], required: true }, // [longitude, latitude]
  },
  totalSlots: { type: Number, required: true, min: 1 },
  hourlyRate: { type: Number, required: true, min: 0 },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
}, { timestamps: true });

// 2dsphere index required for spherical $near geospatial queries
parkingLotSchema.index({ location: '2dsphere' });

export default mongoose.model('ParkingLot', parkingLotSchema);

// server/utils/generateQrToken.js
import { randomBytes } from 'crypto';

const generateQrToken = () => randomBytes(32).toString('hex');
export default generateQrToken;
```